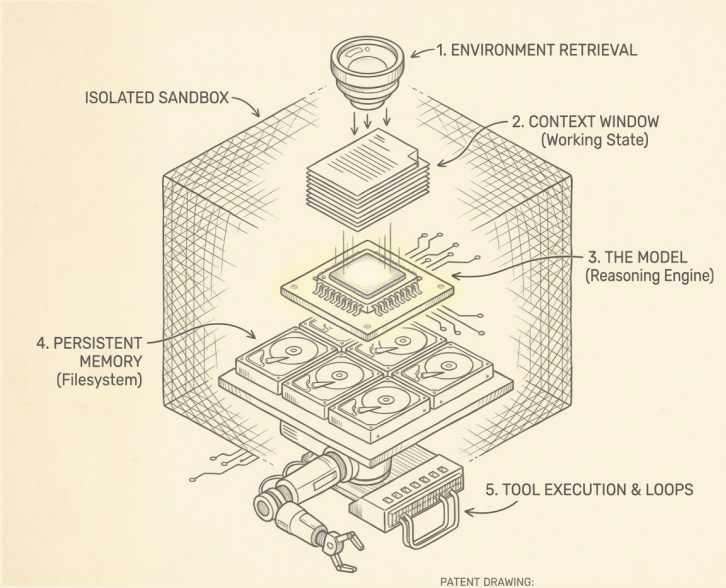
ODSC AI EAST 2026

Engineering the Harness:
A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

https://github.com/rajshah4/harness-engineering



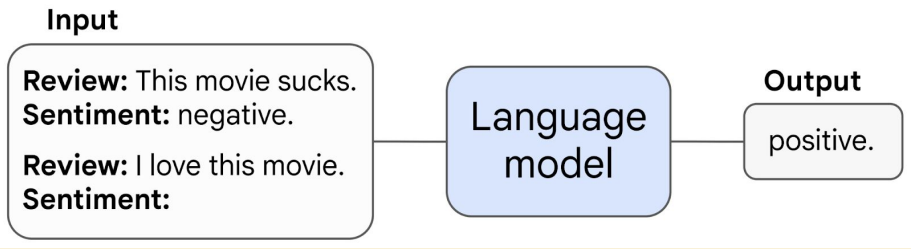
ODSC AI EAST 2026

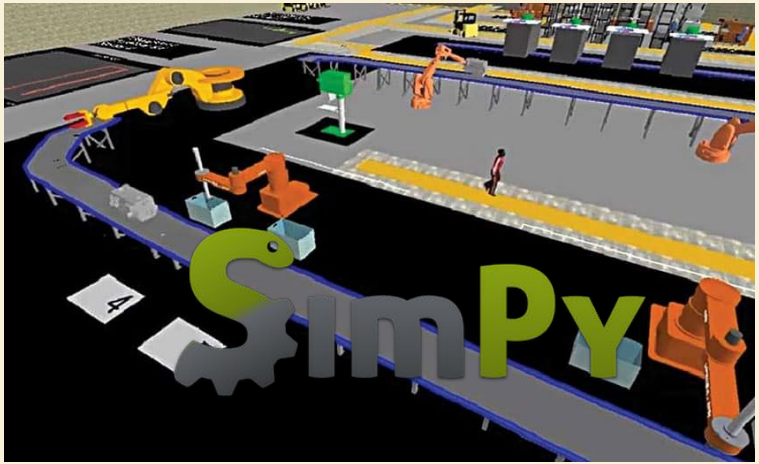
Engineering the Harness:
A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

https://github.com/rajshah4/harness-engineering

The tasks changed.





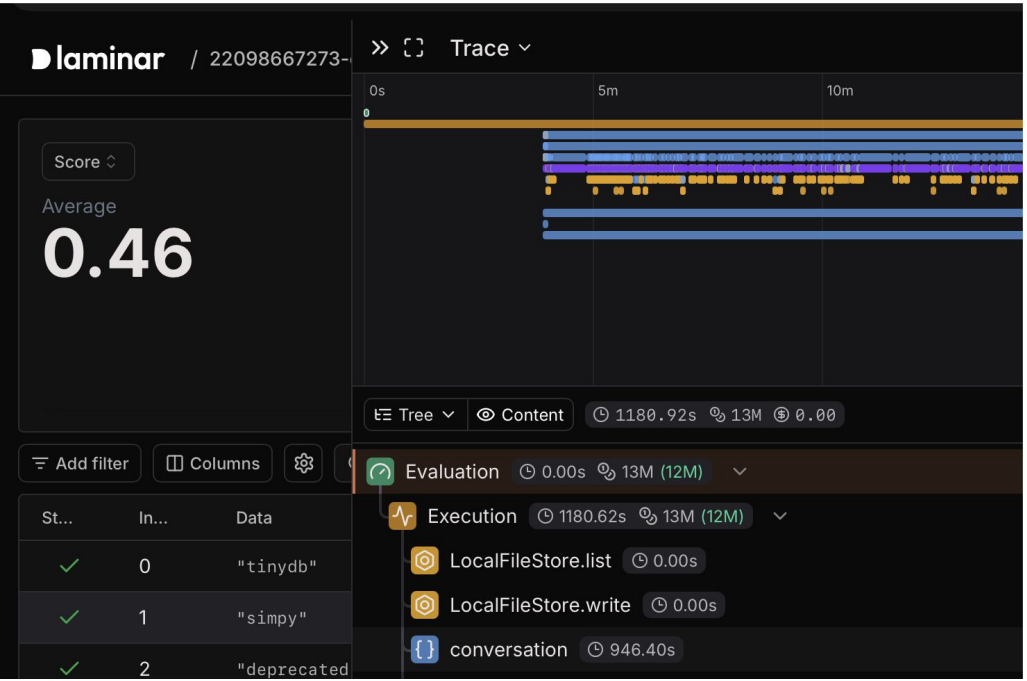
2023
Prompting for sentiment

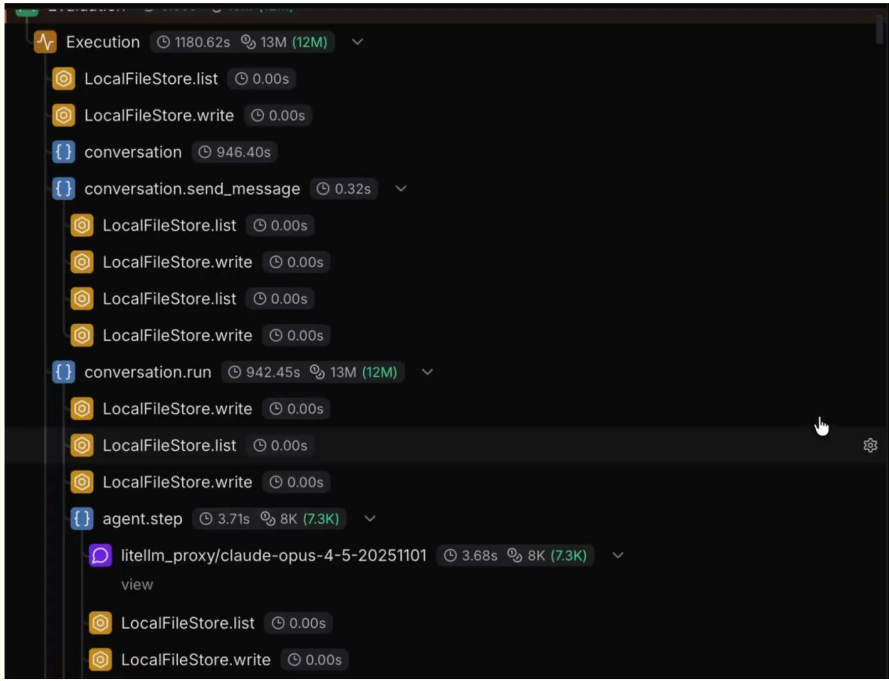
30 tokens - ~0.2 seconds

2026
Fix the bug, here is the code base and test suites

Opus 4.6
12M tokens and 20 minutes

The model isn’t solving the problem. The system is.





13M Tokens for 50k Output! and takes 20 minutes

https://laminar.sh/shared/evals/c97e4a45-8a14-428f-8eac-f77ef6eb75a8

Hi, I'm Rajiv, and this is a masterclass on Harnesses.

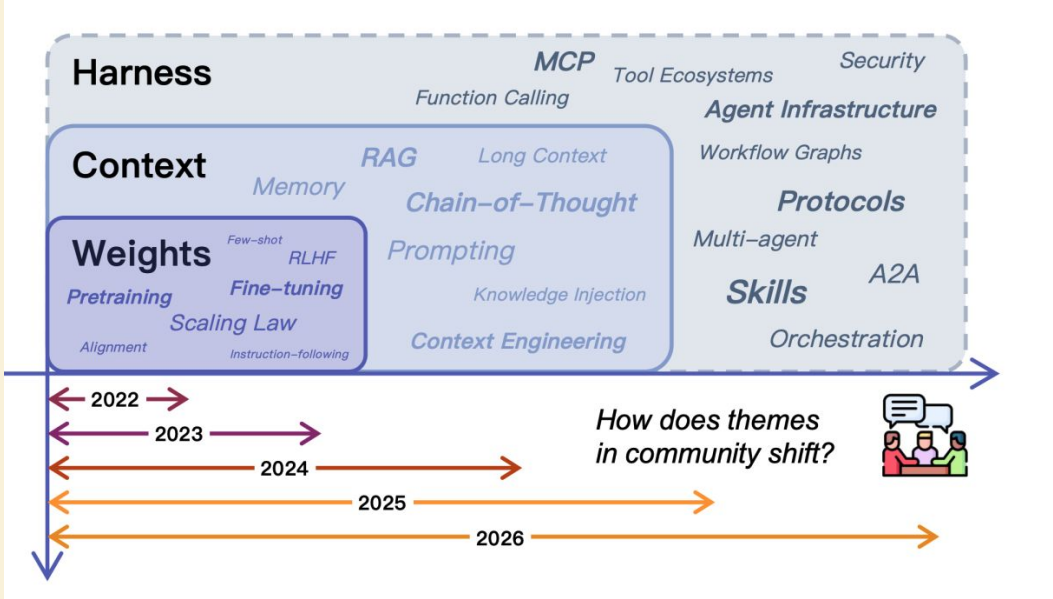
Rajiv Shah

Agentic AI Engineer

OpenHands

Social Media: @rajistics

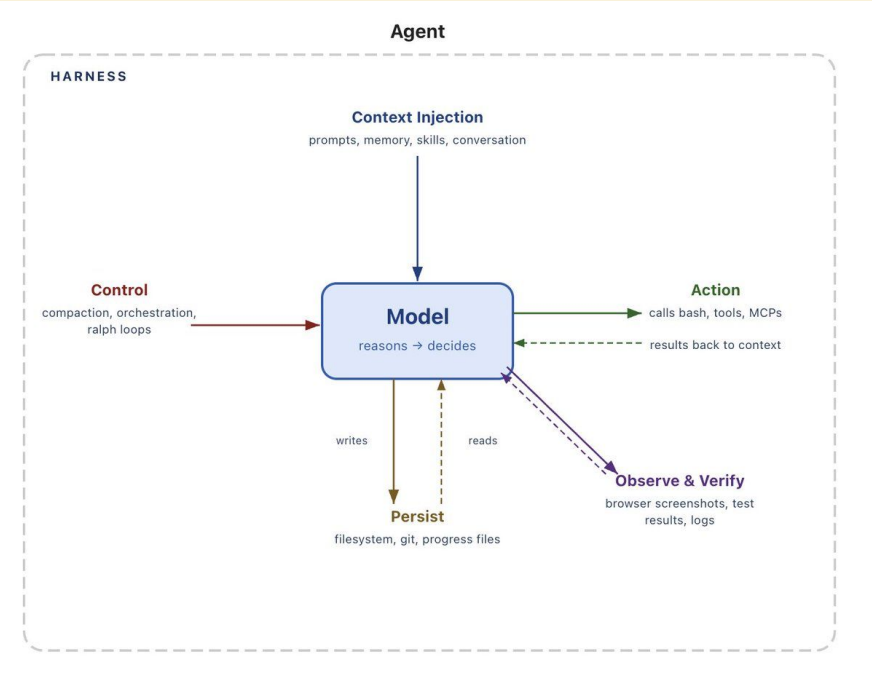
This has been an evolution



LLM Agents

https://arxiv.org/pdf/2604.08224

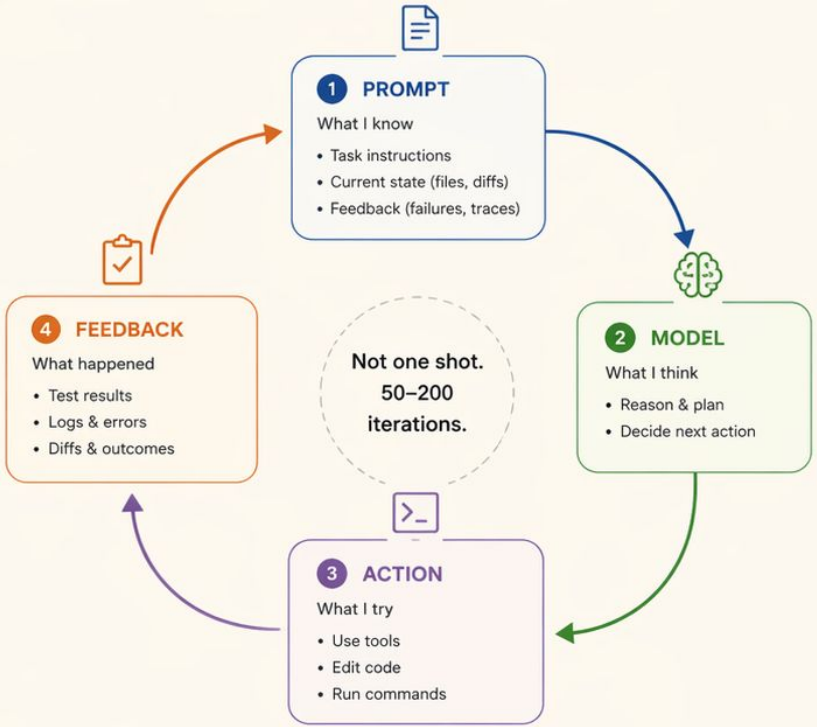
What is in a harness?



https://blog.langchain.com/the-anatomy-of-an-agent-harness/

What a modern coding agent actually looks like.

A continuous loop that turns state into progress.

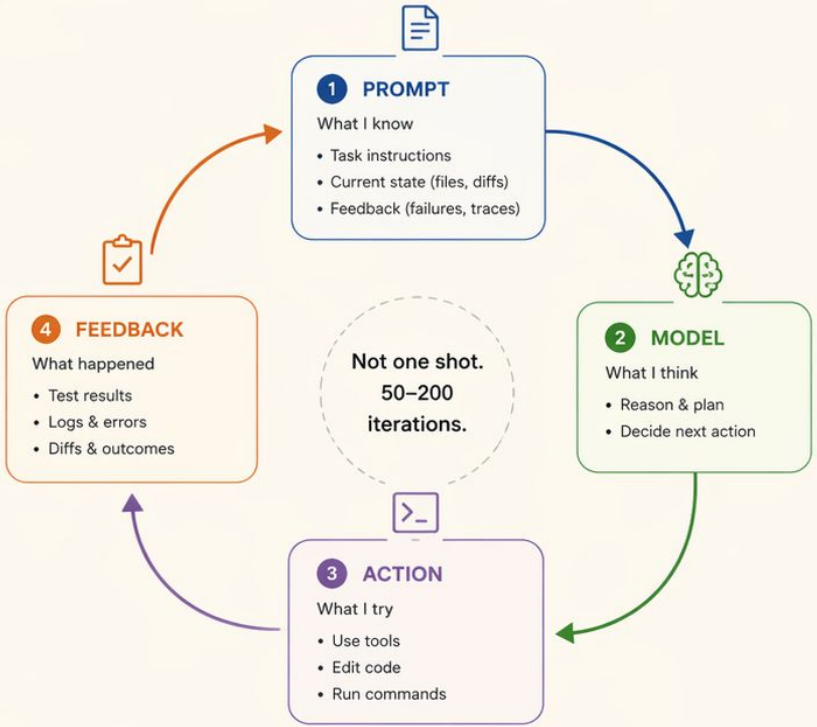


TOOLS (examples)

Run tests | Read code | Edit code | Inspect changes

What a modern coding agent actually looks like.

A continuous loop that turns state into progress.



A harness is everything outside the model.

The harness provides the environment, control, and feedback that make the loop reliable and effective.

A good SDK abstracts the harness for agentic actions

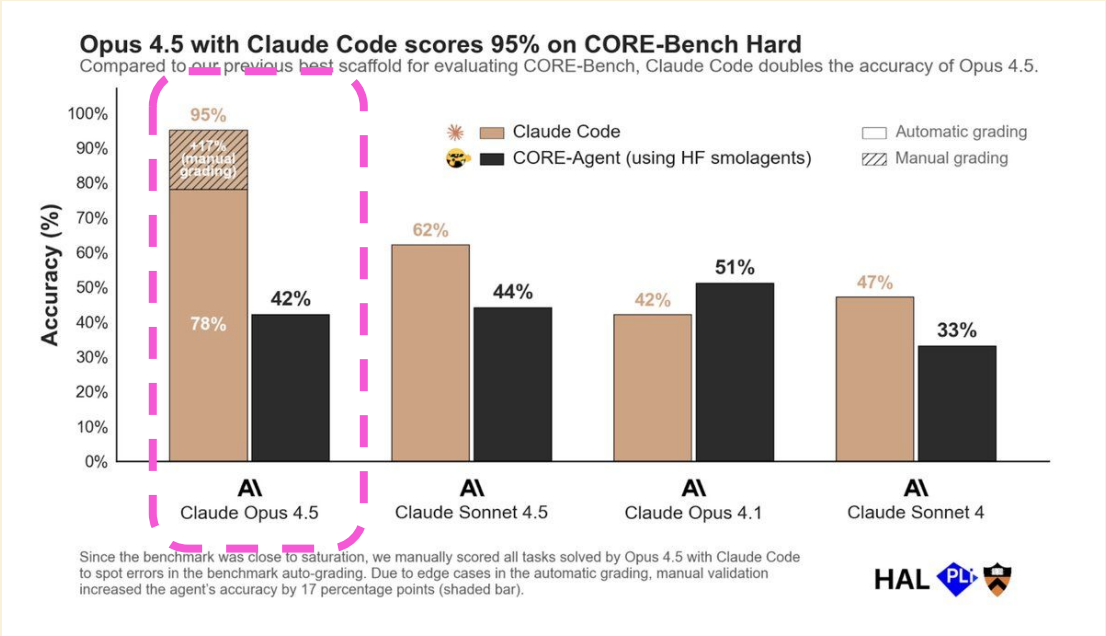
```python
from openhands.agent import Agent
from openhands.runtime import DockerWorkspace
from openhands.conversation import Conversation

# 1. Create workspace (repo environment)
workspace = DockerWorkspace(
    image="swebench:latest",
    working_dir="/workspace/repo"
)

# 2. Initialize agent (the harness entrypoint)
agent = Agent(
    model="anthropic/claude-3",
    tools=["bash", "editor", "git"],
    system_prompt_kwargs={"cli_mode": True},
)

# 3. Wrap in a conversation loop (the core harness)
conversation = Conversation(
    agent=agent,
    workspace=workspace,
    max_iterations=50,
)
```

```python
# 4. Construct the SWE task (benchmark prompt)
task_prompt = f"""
You are given a GitHub issue and a repository.

Issue:
{issue_text}

Your goal:
- Identify the bug
- Modify the code (not tests)
- Run tests to verify the fix
- Produce a valid patch

Work step by step using tools.
"""

# 5. Run the harness loop
conversation.run(task_prompt)

# 6. Extract result (the "work product")
patch = workspace.get_git_diff()
print(patch)
```


https://github.com/OpenHands/software-agent-sdk/

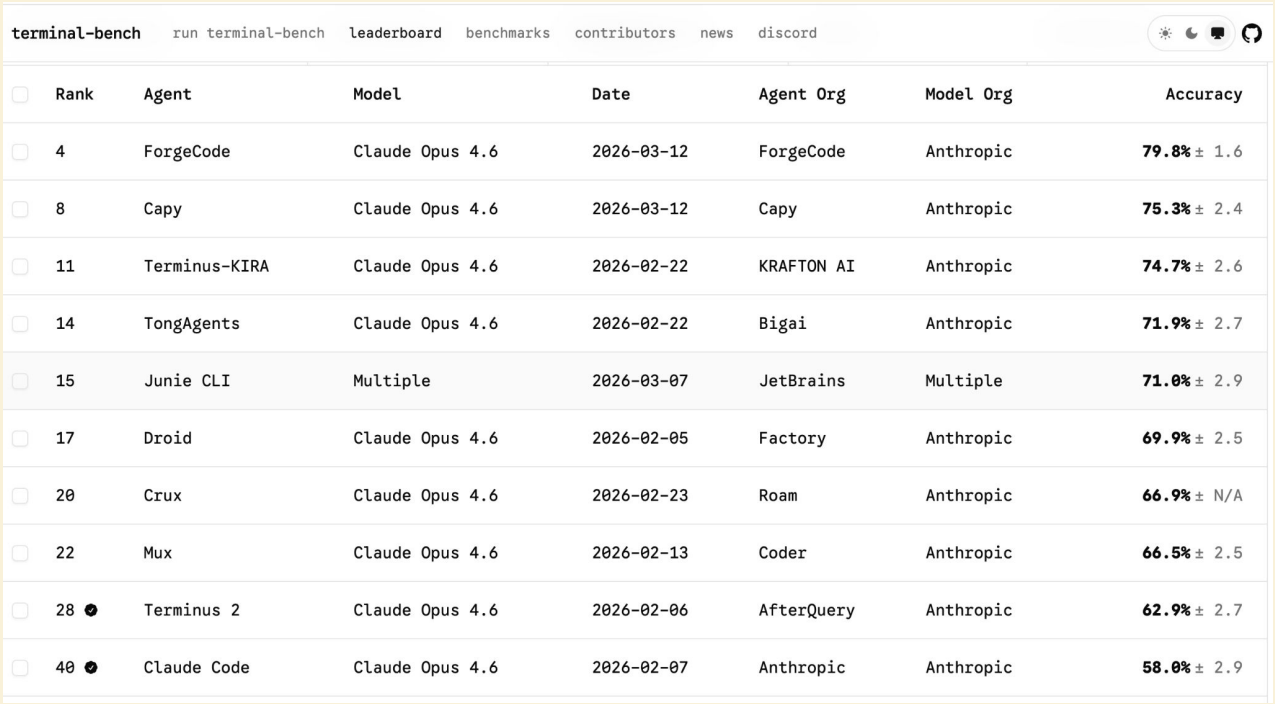
Same model, 2× performance gap

The change:
95% - Claude Code
42% - HF smolagents



https://x.com/sayashk/status/1996334941832089732

Everyone on the leaderboard uses the same model

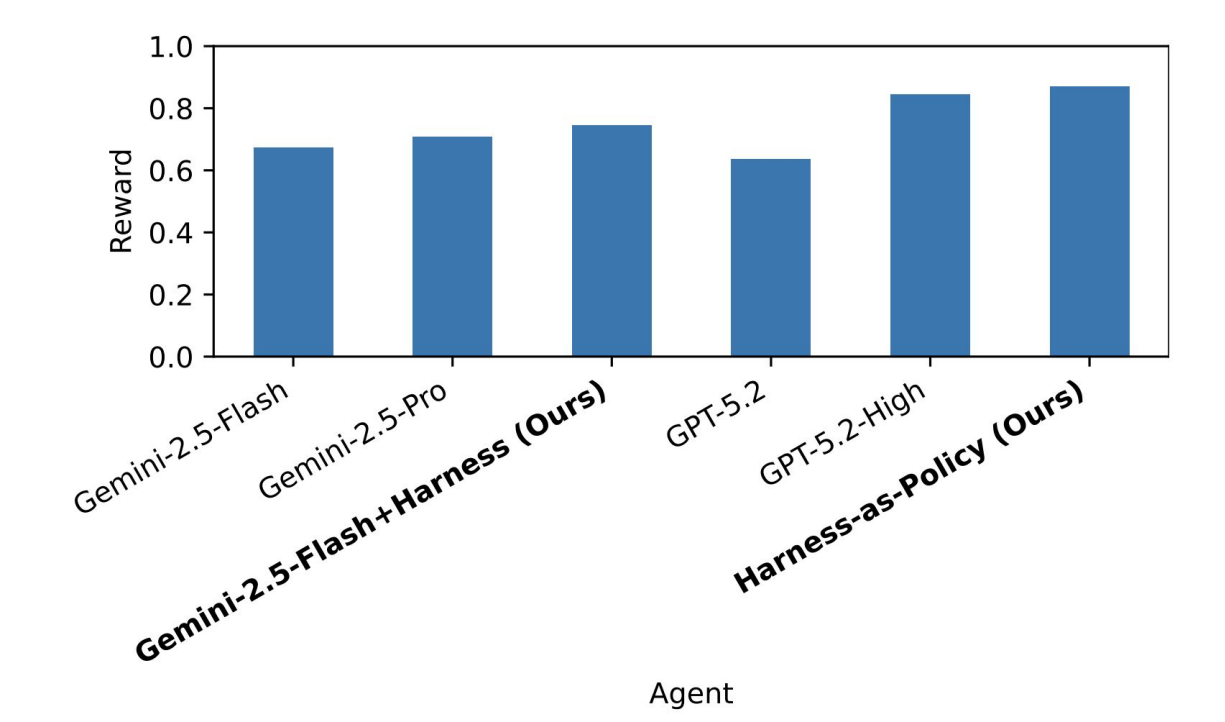


terminal-bench run terminal-bench leaderboard benchmarks contributors news discord

Rank	Agent	Model	Date	Agent Org	Model Org	Accuracy
4	ForgeCode	Claude Opus 4.6	2026-03-12	ForgeCode	Anthropic	**79.8%** ± 1.6
8	Capy	Claude Opus 4.6	2026-03-12	Capy	Anthropic	**75.3%** ± 2.4
11	Terminus-KIRA	Claude Opus 4.6	2026-02-22	KRAFTON AI	Anthropic	**74.7%** ± 2.6
14	TongAgents	Claude Opus 4.6	2026-02-22	Bigai	Anthropic	**71.9%** ± 2.7
15	Junie CLI	Multiple	2026-03-07	JetBrains	Multiple	**71.0%** ± 2.9
17	Droid	Claude Opus 4.6	2026-02-05	Factory	Anthropic	**69.9%** ± 2.5
20	Crux	Claude Opus 4.6	2026-02-23	Roam	Anthropic	**66.9%** ± N/A
22	Mux	Claude Opus 4.6	2026-02-13	Coder	Anthropic	**66.5%** ± 2.5
28	Terminus 2	Claude Opus 4.6	2026-02-06	AfterQuery	Anthropic	**62.9%** ± 2.7
40	Claude Code	Claude Opus 4.6	2026-02-07	Anthropic	Anthropic	**58.0%** ± 2.9

https://www.tbench.ai/

Small model + good harness > big model



AutoHarness: https://arxiv.org/pdf/2508.07995
Meta-Harness: https://yoonholee.com/meta-harness/

What harness do you use?

Harness carries a lot of decisions

Claude Code vs Codex CLI: Harness & Experience Comparison

Dimension	Claude Code Anthropic	Codex CLI OpenAI
Primary Emphasis	Polish, safe defaults, integrated experience	Openness, transparency, flexibility
Core Philosophy	Curated experience with guardrails and conventions	Open and inspectable; user in control of the stack
Project Instructions	CLAUDE.md (hierarchical, auto-detected)	AGENTS.md (hierarchical, auto-detected)
Permissions Model	Fine-grained allow/deny for tools, files, and domains (Bash, Edit, WebFetch, MCP, etc.)	Sandbox + approval modes (Suggest, Auto Edit, Full Auto); network off by default
Sandboxing	Workspace-aware with permission rules and safety checks	Stricter sandbox by default; configurable modes and network control
Extensibility	MCP support, hooks, subagents, skills	MCP support, config profiles, custom commands; open-source CLI surface
Observability & Debugging	More productized; less of the internals exposed	Highly inspectable (open source); easy to see prompts, tool calls, and logs
User Experience	Tight UX, opinionated flows, batteries included	CLI-first, more DIY setup, extremely configurable
Best For	Teams wanting a secure, polished experience out of the box	Power users/teams wanting transparency, customization, and control
Common Friction	Permission friction; hidden harness behavior	Sandbox/config complexity; approval semantics can be surprising
Sweet Spot	Long-lived projects, safety-conscious teams, minimal setup	Automation-heavy workflows, custom integrations, experimentation

Both are powerful. Claude Code optimizes for a polished, safe experience. Codex CLI optimizes for openness and control.

https://fieldjournal.ai/blog/codex-cli-vs-claude-code/?utm_source=chatgpt.com

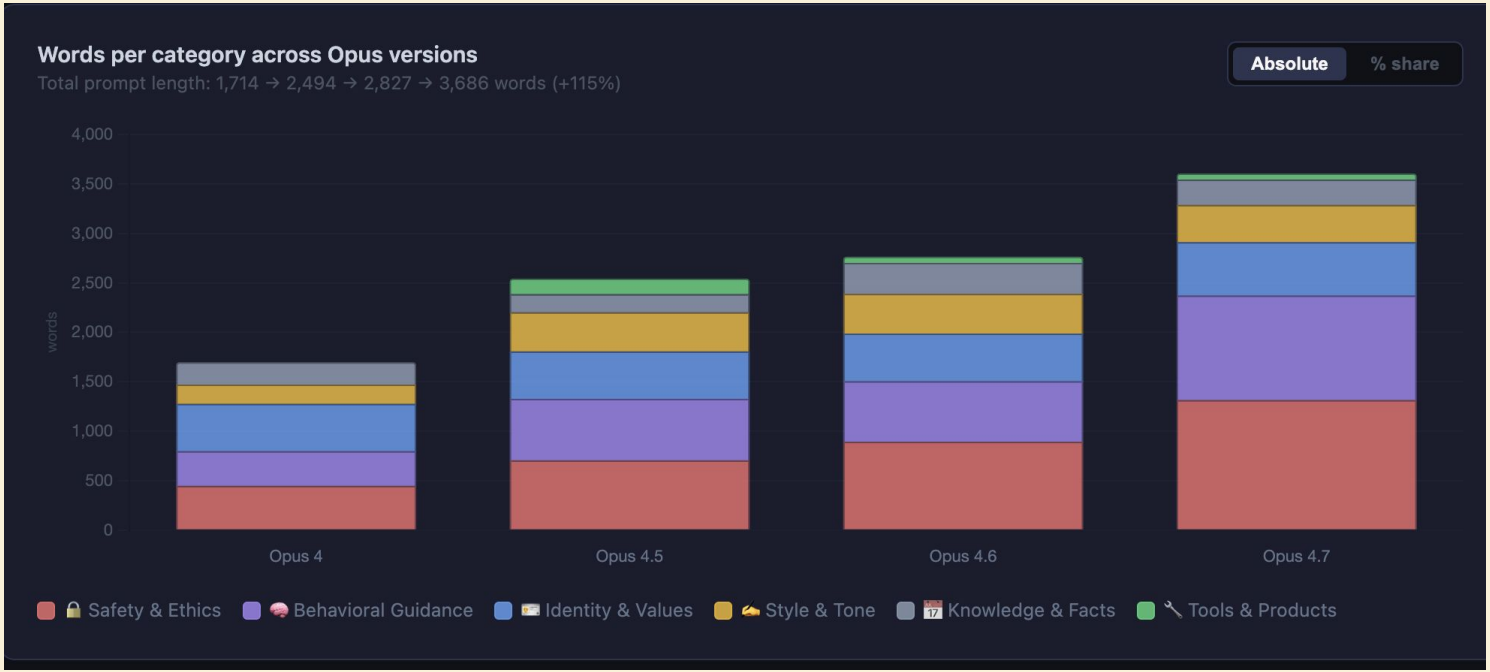
Harnesses must evolve with the models.

- 3 year ago (AutoGen & CrewAI)
- As models improve (e.g., Opus 4.7 context handling), your harness should get simpler.
- Manus rebuilt their harness 5 times

https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus

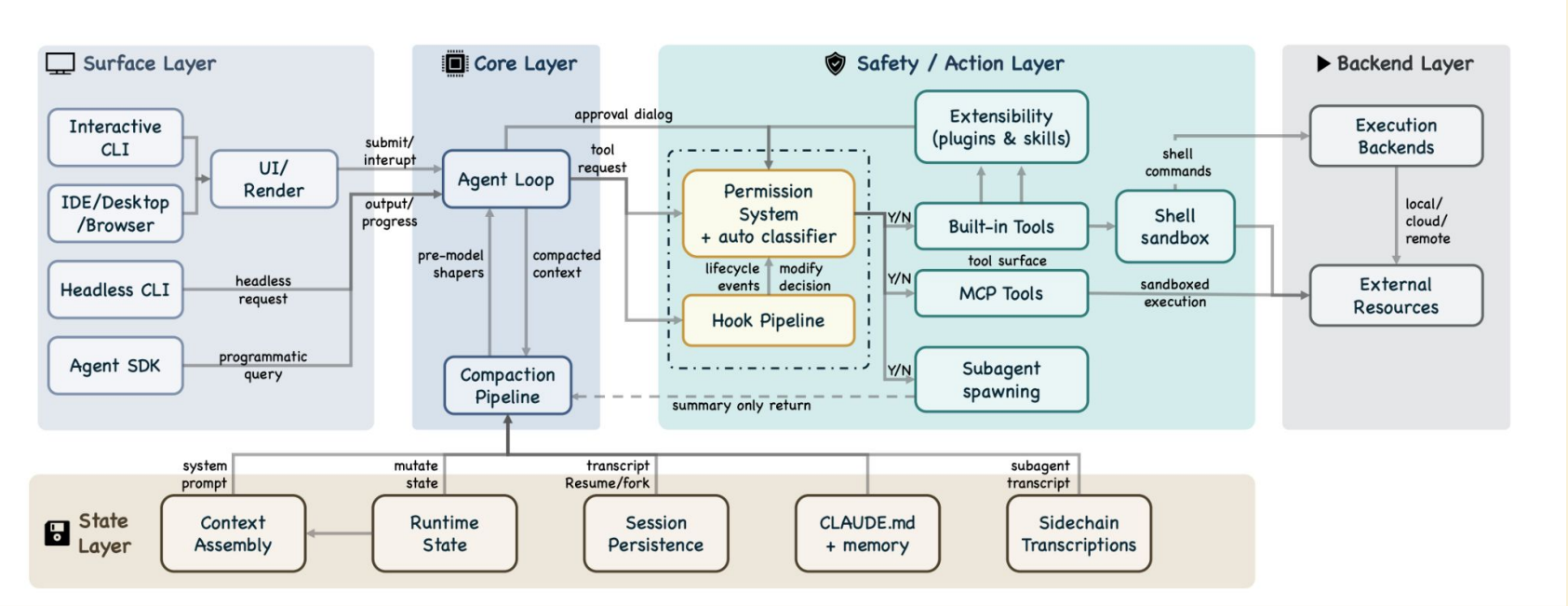
Noticed the trend towards shorter system prompts?

System prompts getting longer!



Raj Harness Repo

Claude Code Harness / Architecture



https://arxiv.org/pdf/2604.14228

Claude Code Bugs

- default reasoning high -> medium
- bug that accidentally evicted thinking blocks on every turn in session was a change to help with cache optimization
- system prompt change to reduce verbosity which reduced code quality

https://www.anthropic.com/engineering/april-23-postmortem

What model are you using?

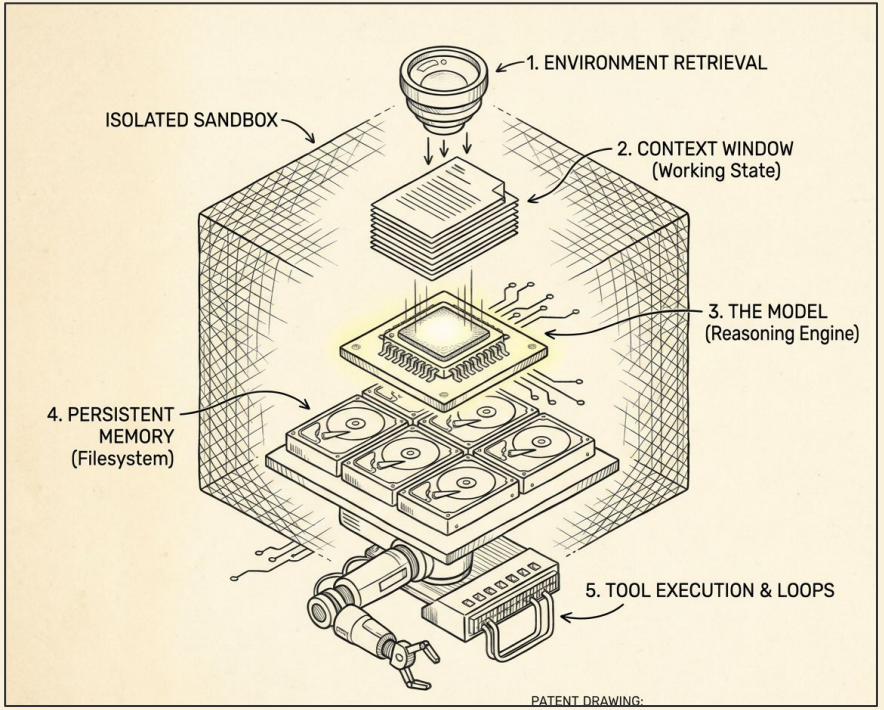
Who owns the harness?

- Stateless chat → easy to switch
- Stateful agents → locked in

The harness owns your context, memory, and tools.
Pick accordingly.

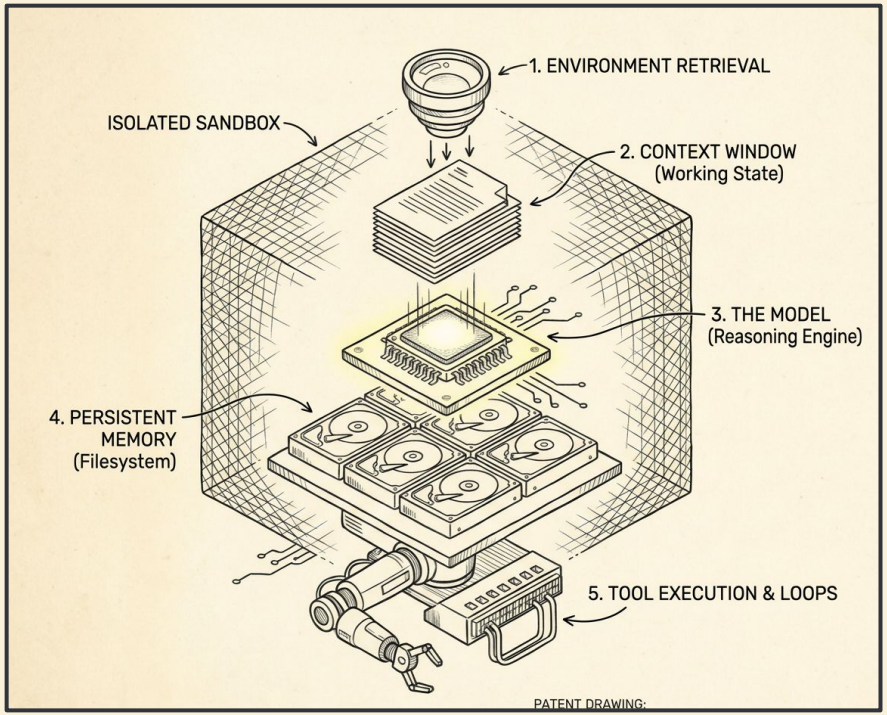
The 5 Levers of Harness Engineering

- The Model
- Retrieval
- Memory & State
- Agentic Loops & Tool Use
- System Architecture



Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



The three modes of Agentic Retrieval.

BM25
Keyword-based retrieval

Language Models
Semantic meaning with embeddings

Agentic Search
Dynamic using LLM Reasoning

The Baseline: Scaling grep.

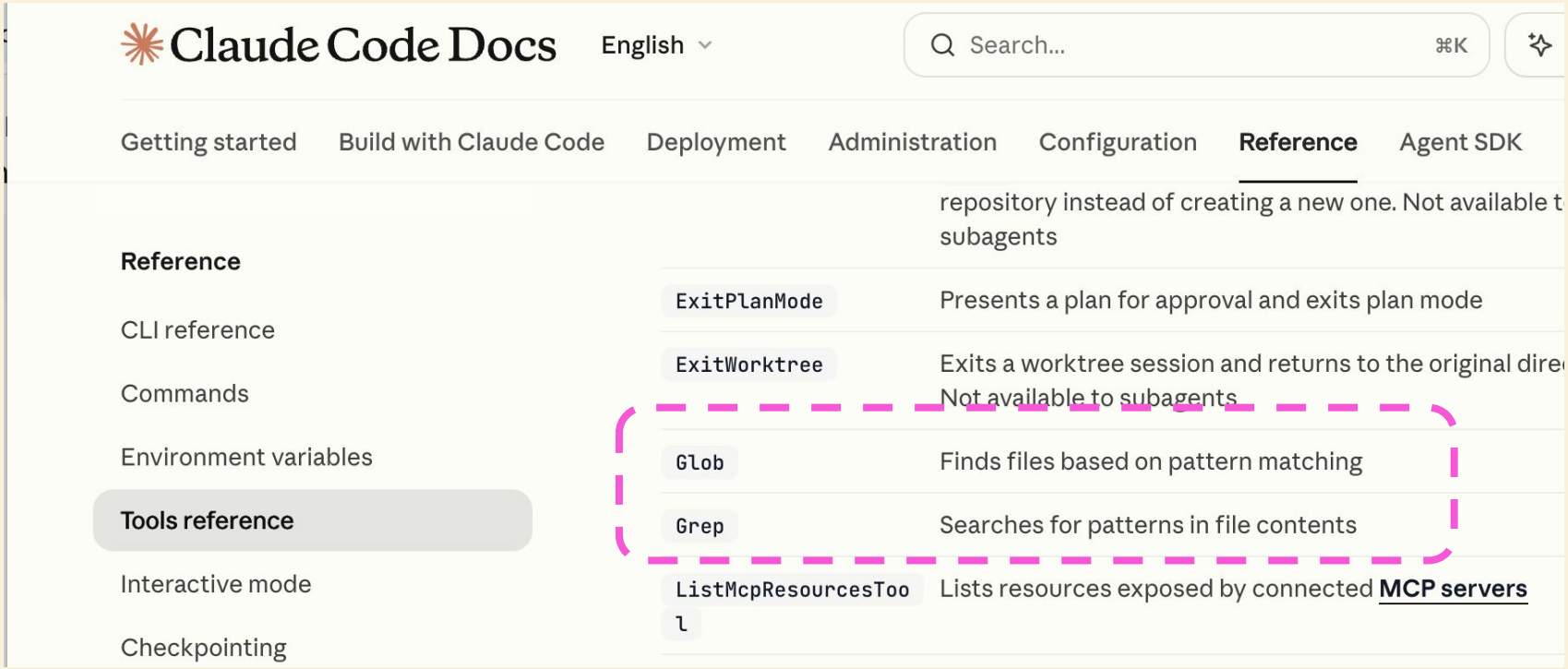
- Keyword precision
- Sub-second latency
- Battle-tested

Inverted Indices (BM25) make grep instant.

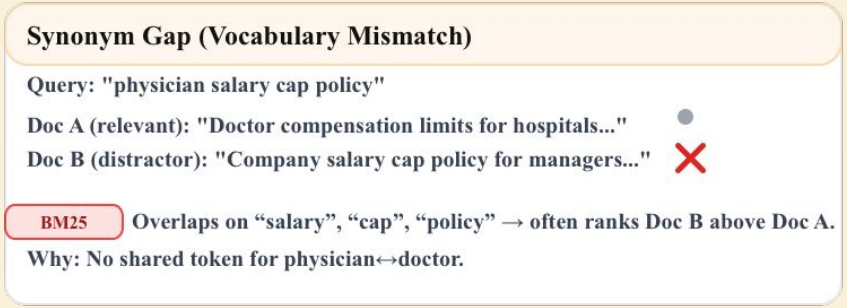
You don't need a vector database to search code.
BM25 is blazing fast.

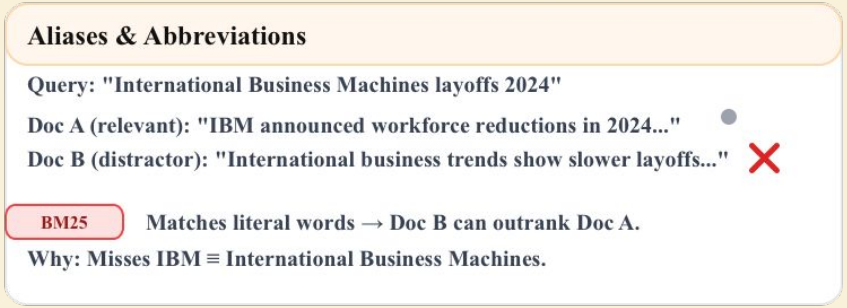
N_docs	Linear/Grep (s)	Inverted Index	BM25 (s)
1000	3.468	0.005	0.028
3000	10.188	0.014	0.097
6000	20.608	0.025	0.24
9000	30.092	0.061	0.36

State-of-the-Art Coding Agents rely on Lexical Search.



Where Lexical breaks down: The Synonym Gap.

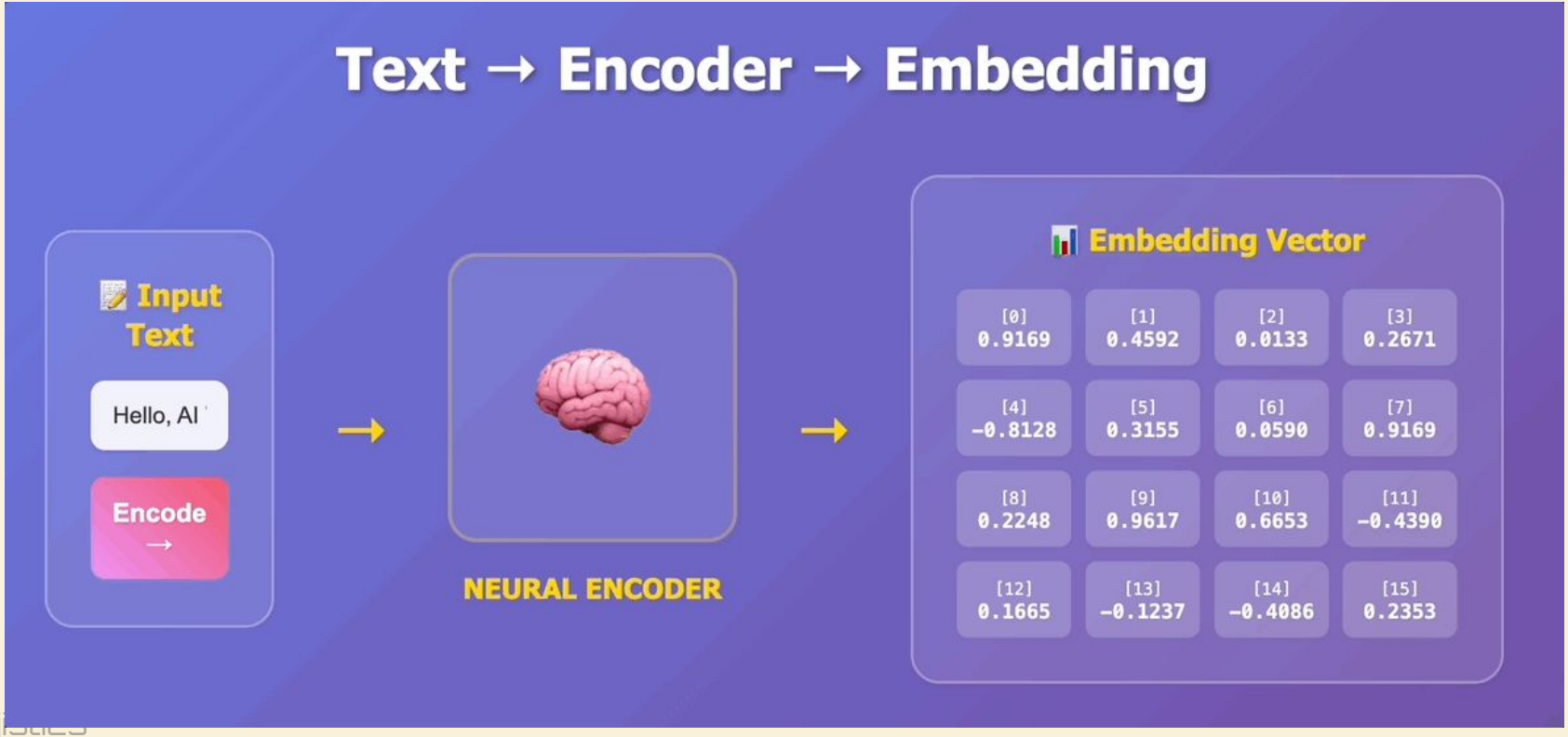




Takeaway:

- If you have keyword-heavy queries and need sub-second response → text search might be sufficient

Embeddings solve for meaning.

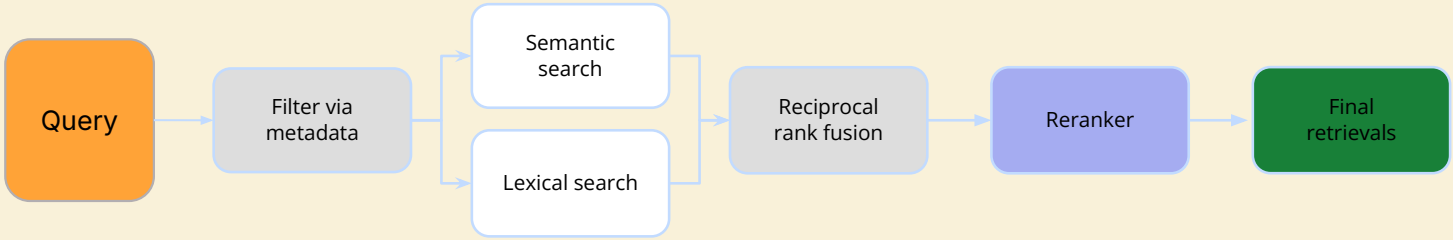


Semantic search is still required for massive codebases.

- **Cursor**
  - Achieving on average 12.5% higher accuracy
  - Producing code changes that are more likely to be retained in codebases.

https://cursor.com/blog/semsearch

The Status Quo: Hybrid Search.



Prioritizing n=100

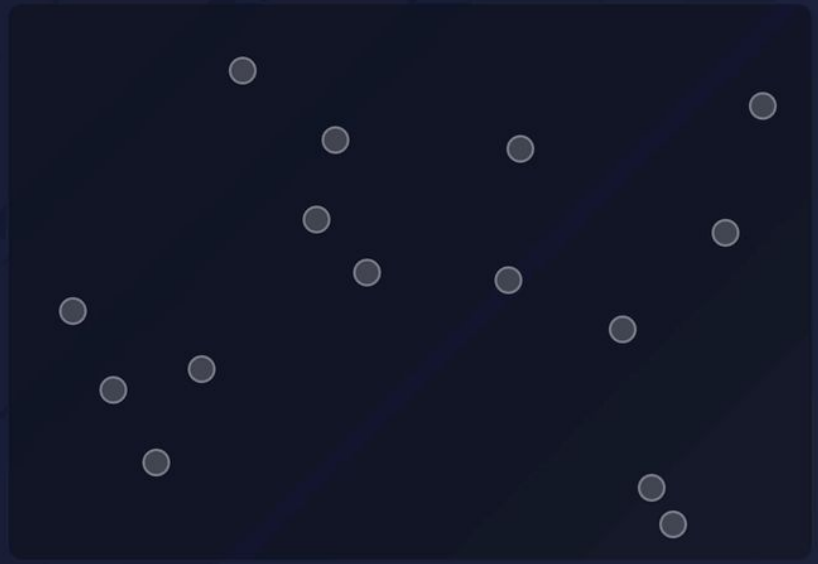
Prioritizing n=10

Chroma: https://x.com/trychroma/status/1983625513244750304

Search Strategy Comparison

Watch how different approaches explore the solution space

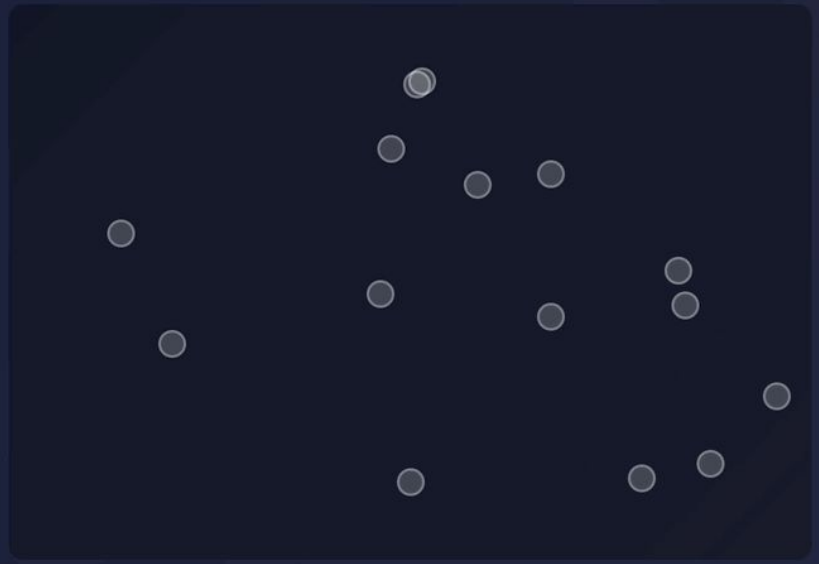
Traditional RAG



Ready to search...

0	0ms
Queries Made	Time Taken

Agentic RAG



Ready to explore...

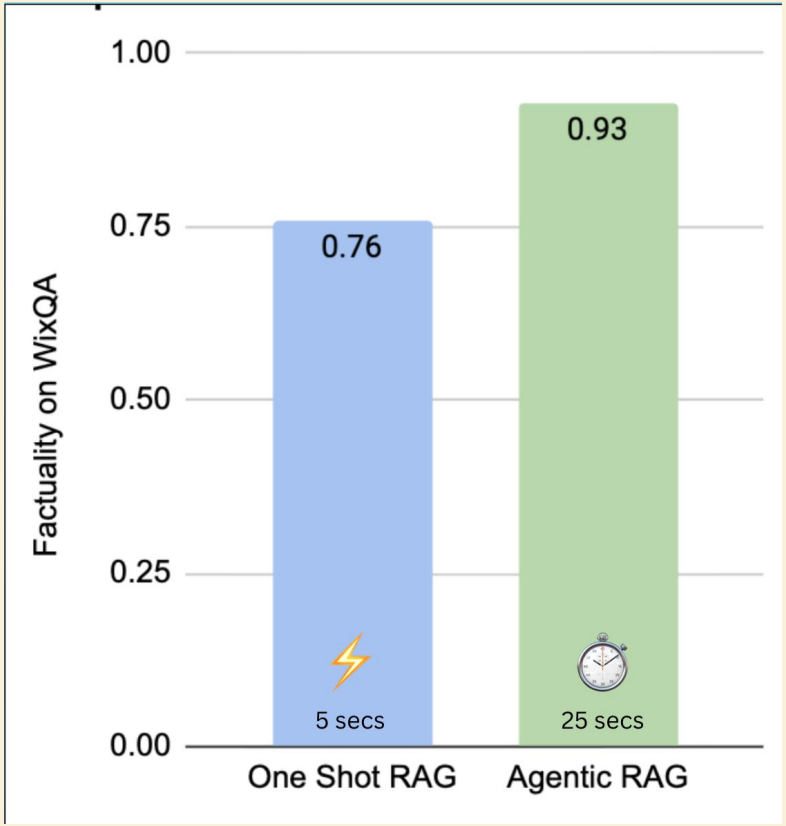
0	0ms
Queries Made	Time Taken

Agentic Search tradeoffs?

Agentic Search trades latency for massive accuracy.

Pick:
- Accuracy
- Latency

(5s versus 25s)



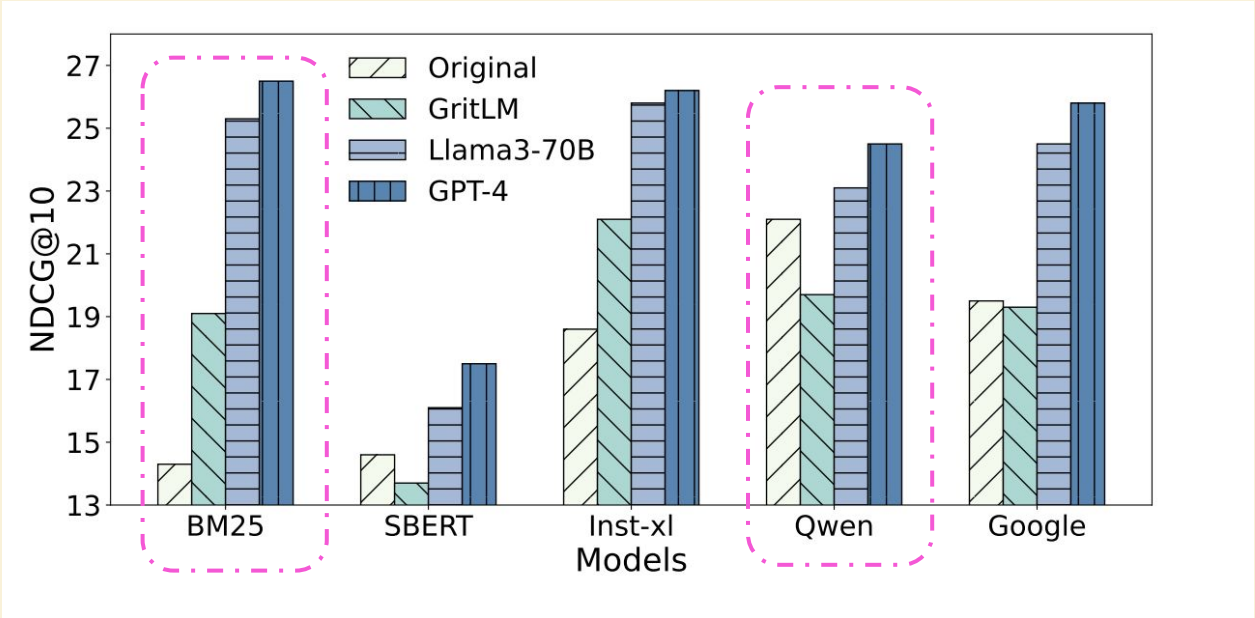
WIxQA: https://arxiv.org/abs/2505.08643

Agentic BM25 beats Semantic Search on reasoning.

Querying with LLM using BM25!!

Agentic Search

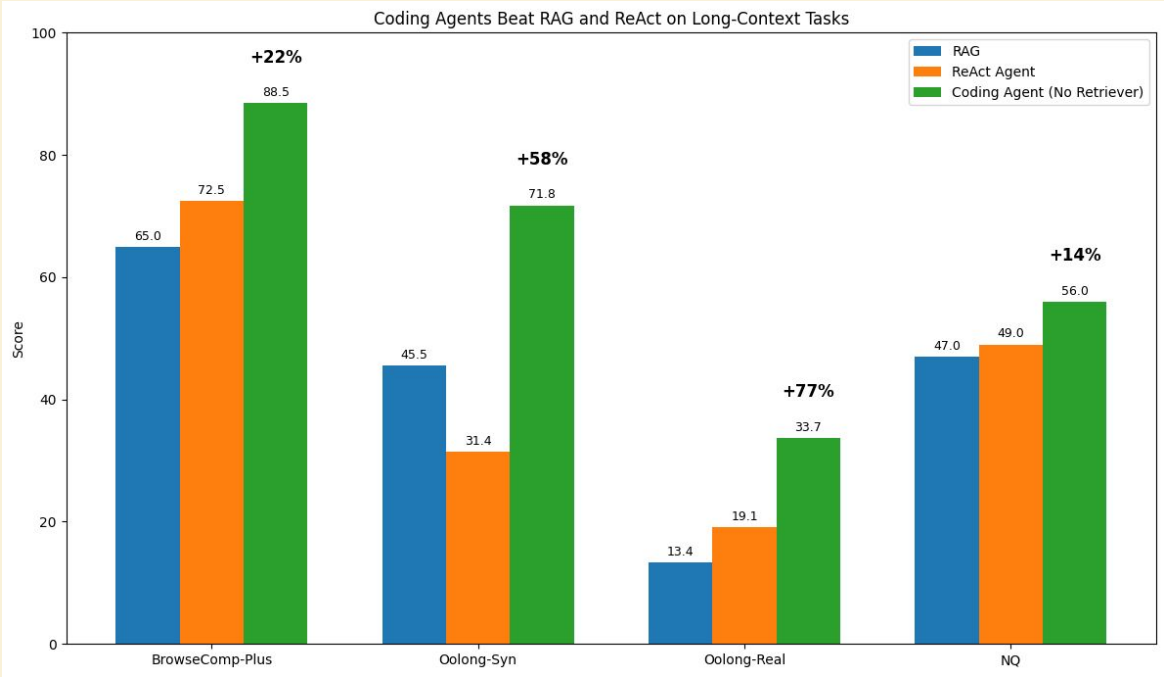
LLMs know synonyums



BRIGHT: https://arxiv.org/pdf/2407.12883

Coding agents are really good at long Context

- Coding agents with file access are very good



https://arxiv.org/pdf/2603.20432

Don't chunk if you don't have to: Whole files > RAG.

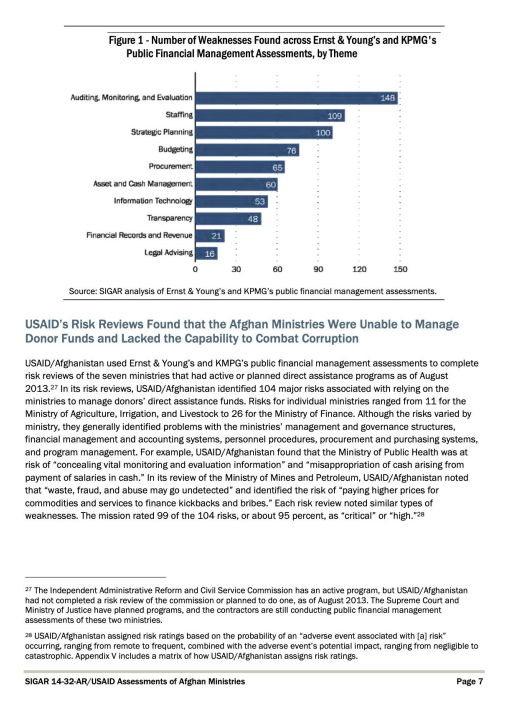


Figure 1 - Number of Weaknesses Found across Ernst & Young's and KPMG's Public Financial Management Assessments, by Theme

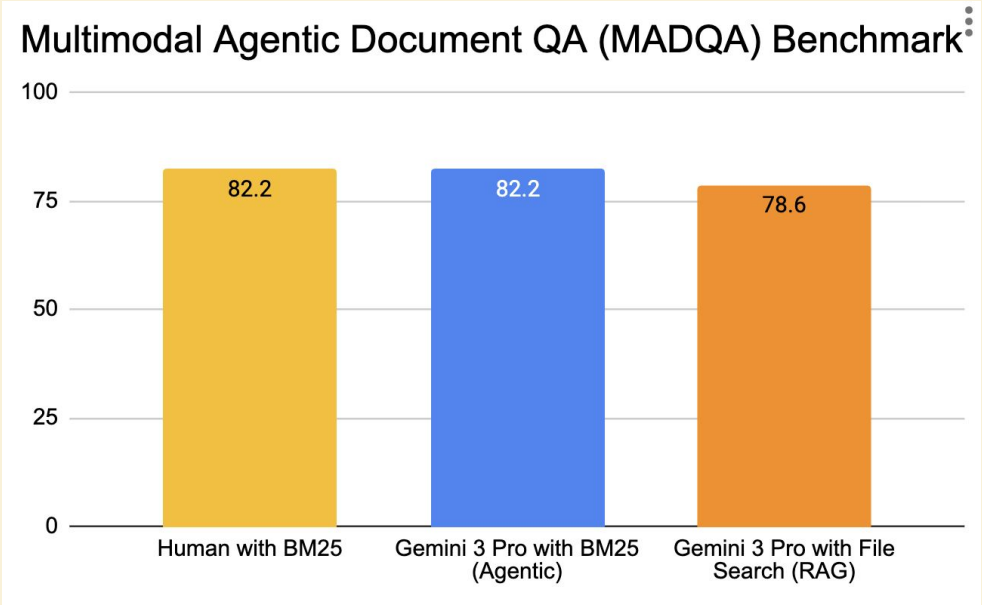
Source: SIGAR analysis of Ernst & Young's and KPMG's public financial management assessments.

USAID's Risk Reviews Found that the Afghan Ministries Were Unable to Manage Donor Funds and Lacked the Capability to Combat Corruption

USAID/Afghanistan used Ernst & Young's and KMPG's public financial management assessments to complete risk reviews of the seven ministries that had active or planned direct assistance programs as of August 2013.[27] In its risk reviews, USAID/Afghanistan identified 104 major risks associated with relying on the ministries to manage donors' direct assistance funds. Risks for individual ministries ranged from 11 for the Ministry of Agriculture, Irrigation, and Livestock to 26 for the Ministry of Finance. Although the risks varied by ministry, they generally identified problems with the ministries' management and governance structures, financial management and accounting systems, personnel procedures, procurement and purchasing systems, and program management. For example, USAID/Afghanistan found that the Ministry of Public Health was at risk of "concealing vital monitoring and evaluation information" and "misappropriation of cash arising from payment of salaries in cash." In its review of the Ministry of Mines and Petroleum, USAID/Afghanistan noted that "waste, fraud, and abuse may go undetected" and identified the risk of "paying higher prices for commodities and services to finance kickbacks and bribes." Each risk review noted similar types of weaknesses. The mission rated 99 of the 104 risks, or about 95 percent, as "critical" or "high."[28]

[27] The Independent Administrative Reform and Civil Service Commission has an active program, but USAID/Afghanistan had not completed a risk review of the commission or planned to do one, as of August 2013. The Supreme Court and Ministry of Justice have planned programs, and the contractors are still conducting public financial management assessments of these two ministries.

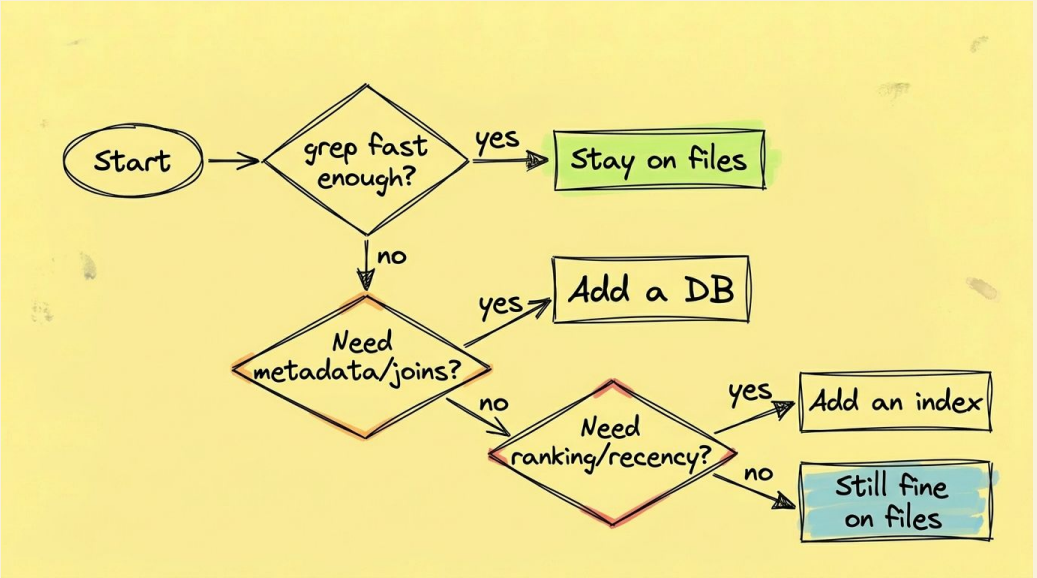
[28] USAID/Afghanistan assigned risk ratings based on the probability of an "adverse event associated with [a] risk" occurring, ranging from remote to frequent, combined with the adverse event's potential impact, ranging from negligible to catastrophic. Appendix V includes a matrix of how USAID/Afghanistan assigns risk ratings.



https://arxiv.org/pdf/2603.12180

So when should you move to a database

Files stay the source of truth. Index on top when query cost bites.

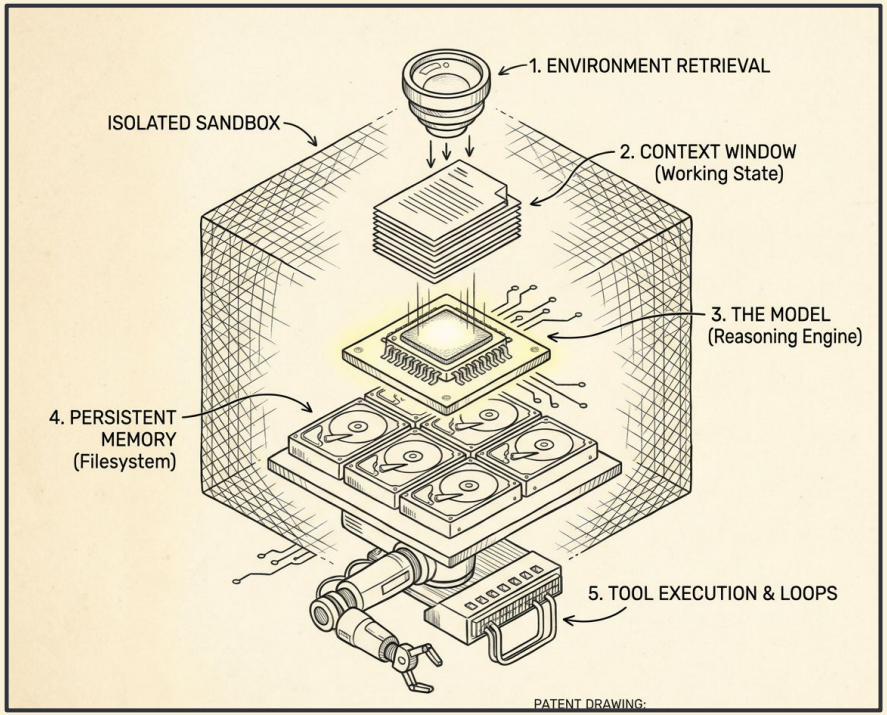


Design rules for you

- **Lexical for Small Set of Files:** BM25 / grep is your baseline.
- **Add Semantics for Speed:** Only if you suffer from vocabulary mismatch.
- **Loop it if Accuracy is Key:** Ultimately, put the retrieval inside an iterative agentic loop.

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



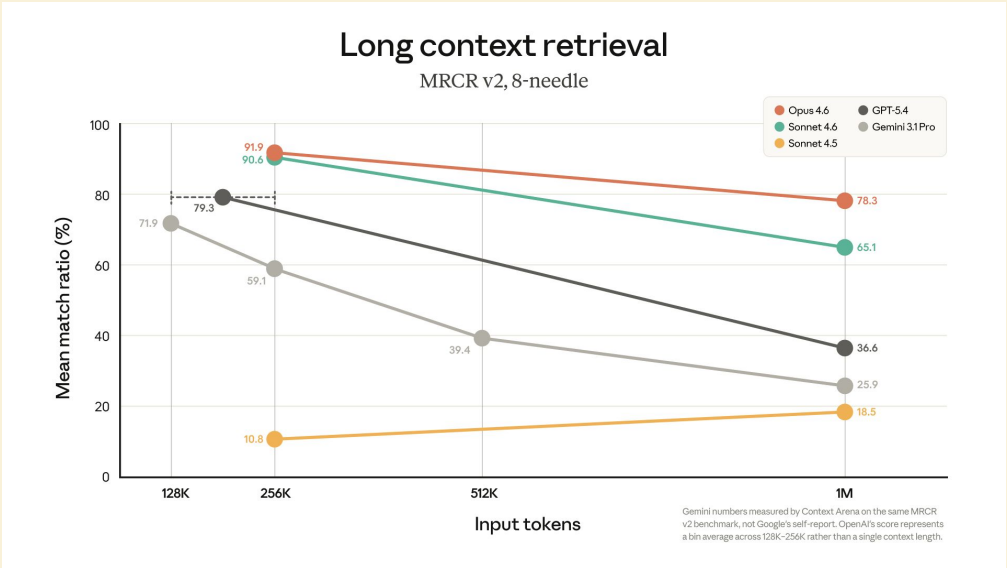
Memory & State

Agents usually fail from losing the thread, not from hitting the token limit

Are you excited about 10M Context Windows?

1M Context Windows are a trap.

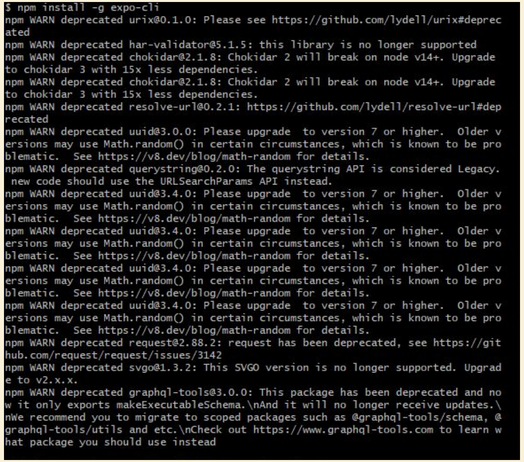
- 500 page multimodal PDF
- 20k rows of data
- 100k lines / 5 MB

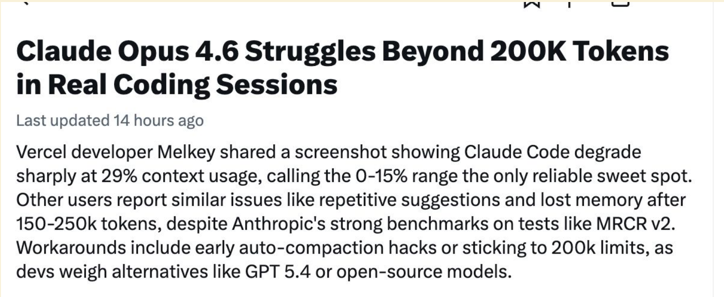


Models degrade when you stuff them full.

https://claude.com/blog/1m-context-ga

Verbose tool traces crowd out the goal.





Claude Opus 4.6 Struggles Beyond 200K Tokens in Real Coding Sessions

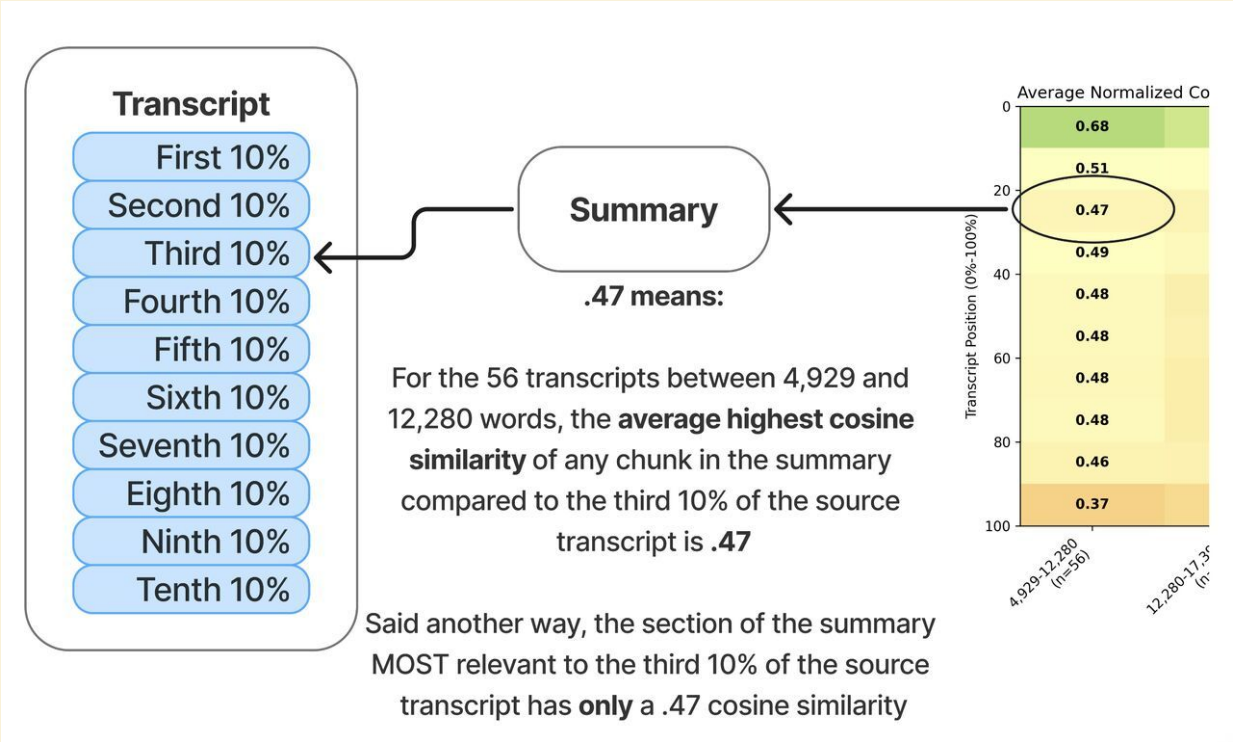
Last updated 14 hours ago

Vercel developer Melkey shared a screenshot showing Claude Code degrade sharply at 29% context usage, calling the 0-15% range the only reliable sweet spot. Other users report similar issues like repetitive suggestions and lost memory after 150-250k tokens, despite Anthropic's strong benchmarks on tests like MRCR v2. Workarounds include early auto-compaction hacks or sticking to 200k limits, as devs weigh alternatives like GPT 5.4 or open-source models.

When 100s of lines of warning push the real goal out of the agent's context window.

@rajistics

Key facts disappear inside long model inputs

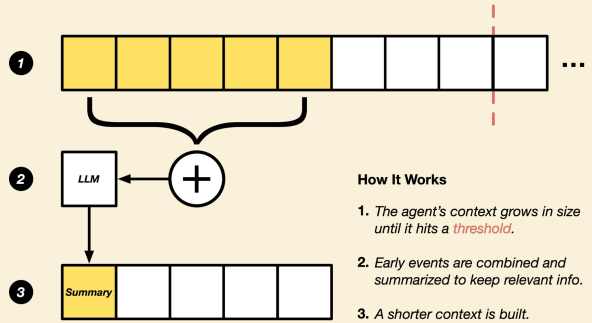


https://www.linkedin.com/posts/sinan-ozdemir_agenticai-llm-rag-ugcPost-7428125462201102336-B7Br

Three layers of Memory

- **Layer 1 - Active Context:** What is in the prompt right now.
- **Layer 2 - Working State:** Plans, TODOs, scratchpads.
- **Layer 3 - Durable Memory:** Skills and reusable workflows.

Layer 1: Fixing Active Context



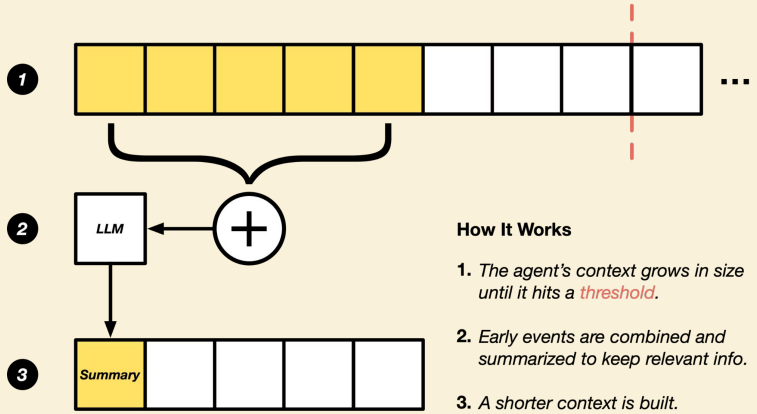
Reset:

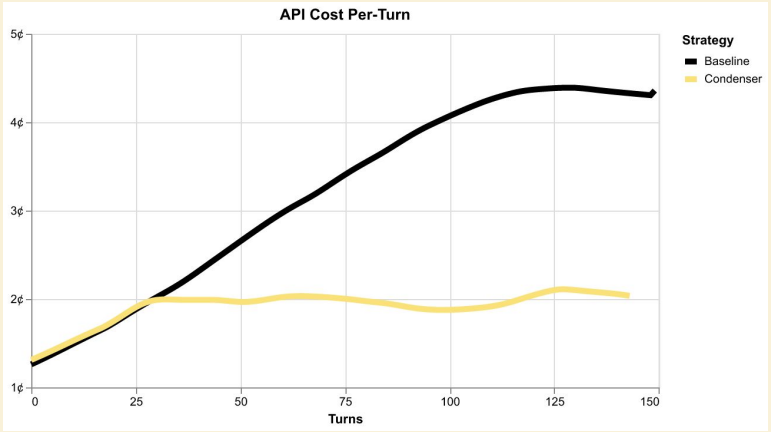
- Clear the window entirely.
- Refill it with only the original instructions and critical artifacts.

Compacting:

- Summarize older turns but keep recent turns intact

Layer 1: Compacting from OpenHands



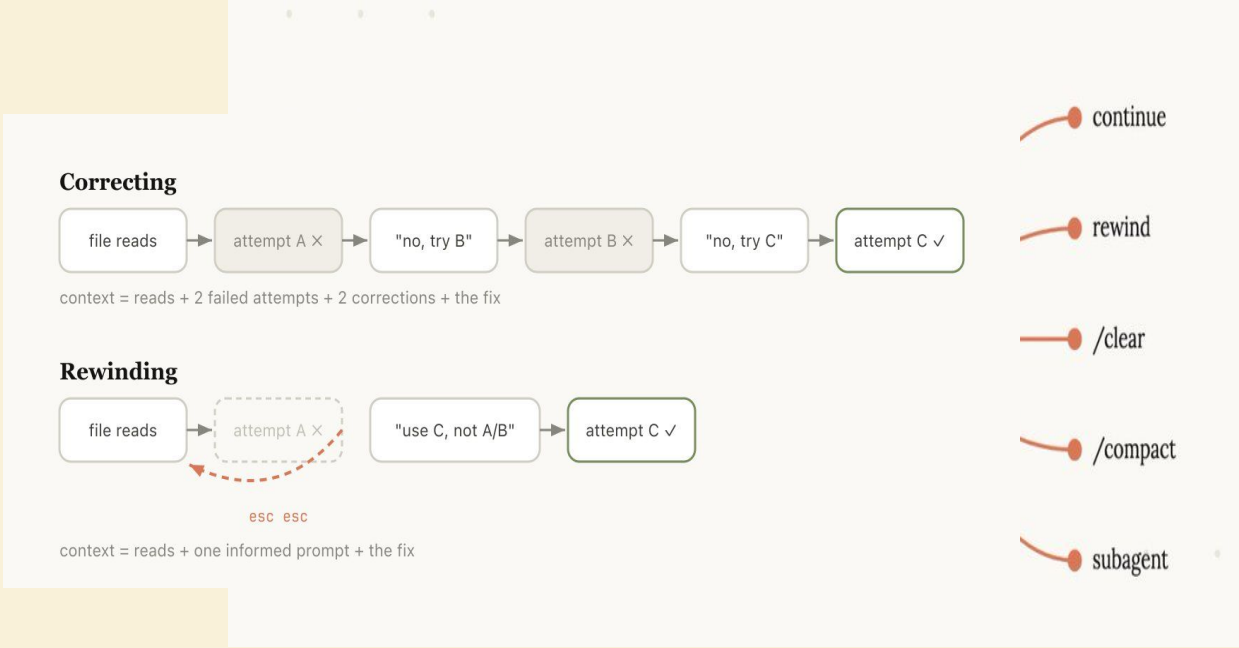


💰 Up to 2x per-turn API cost reduction
⚡ Consistent response times in long sessions
🧠 Equivalent (or better!) performance on software engineering tasks

https://openhands.dev/blog/openhands-context-condensation-for-more-efficient-ai-agents
ACON: https://arxiv.org/html/2510.00615v1

Getting the most of a 1M Context Windows

Techniques from Anthropic on managing your context window



https://x.com/trq212/status/2044548257058328723
https://code.claude.com/docs/en/agent-sdk/file-checkpointing

Layer 2: Working State (The Golden Rule)

Files make better memory than chat.

Write your plan to a .md file in the workspace rather than holding it in prompt memory.

Task Breakdown

Phase 1 — Project Setup

- [ ] Initialise Node.js project (`npm init`)
- [ ] Install dependencies: `express`, `prisma`, `jsonwebtoken`, `bcrypt`, `dotenv`
- [ ] Configure `.env` with `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`
- [ ] Set up Prisma schema with `User` model

Phase 2 — Database Layer

- [ ] Define `User` model in `schema.prisma`
  - Fields: `id`, `email`, `passwordHash`, `createdAt`, `refreshToken`
- [ ] Run initial migration: `npx prisma migrate dev`
- [ ] Create `db.ts` Prisma client singleton

Phase 3 — Auth Logic

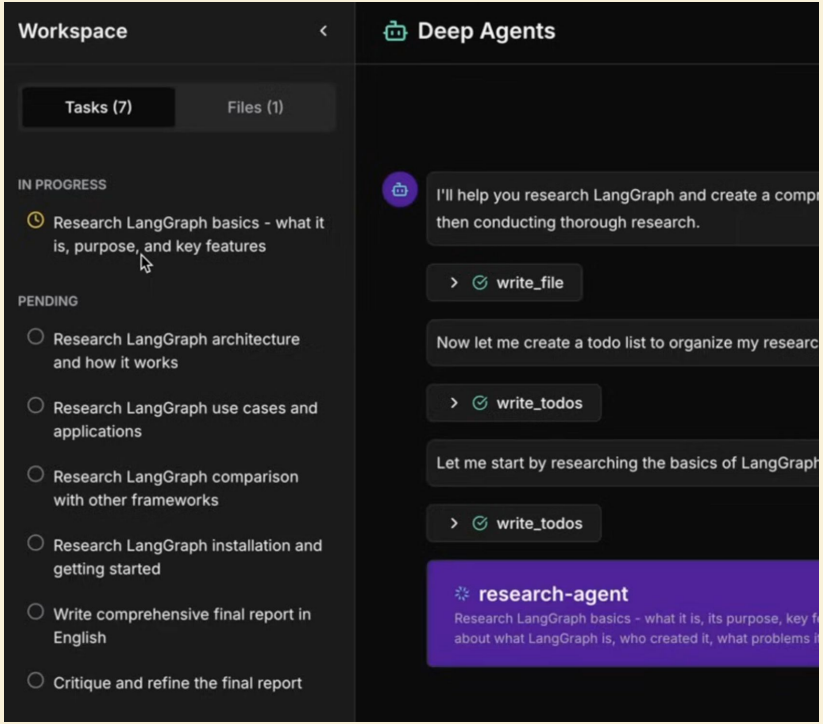
- [ ] `auth/hash.ts` — `hashPassword()` and `verifyPassword()` helpers
- [ ] `auth/tokens.ts` — `generateAccessToken()`, `generateRefreshToken()`, `verifyToken()`
- [ ] Token expiry: access = 15 min, refresh = 7 days

Phase 4 — API Routes

- [ ] `POST /auth/register` — validate input, hash password, create user
- [ ] `POST /auth/login` — verify credentials, return token pair
- [ ] `POST /auth/refresh` — validate refresh token, issue new access token
- [ ] `POST /auth/logout` — invalidate refresh token in DB
- [ ] `GET /me` — protected route returning current user profile

Deep Agents rely on external plans.

LangChain's Deep Agents
uses a write_todos tool to write
out plans for agent tasks



https://www.youtube.com/watch?v=geTtqyFnyHA

Specs for software development

🎯 Goals: what "done" looks like
🚧 Constraints: the box the agent must stay inside
✅ Acceptance: the test the agent runs against itself

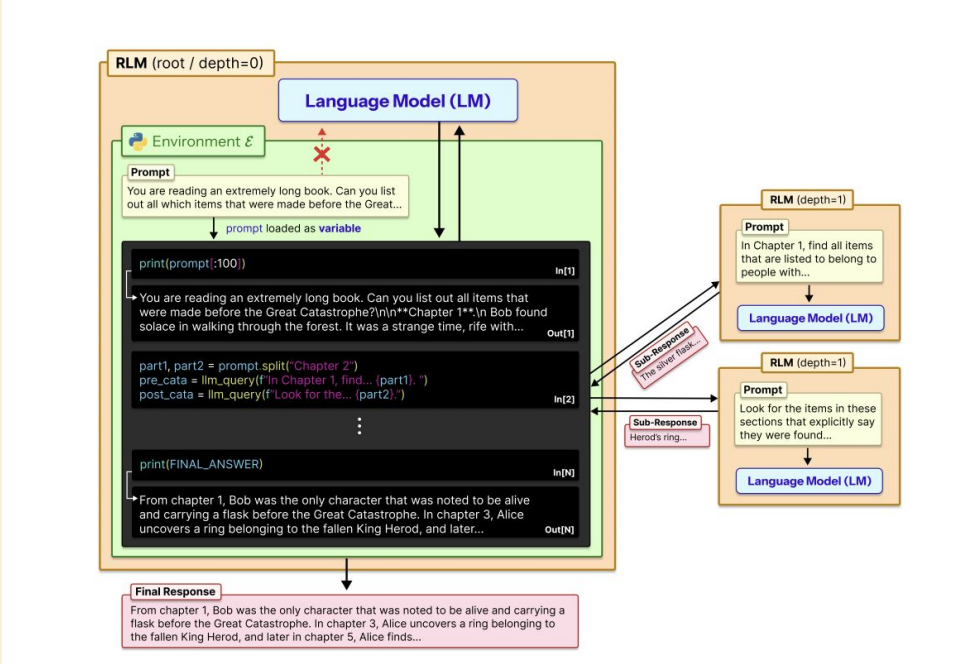
```
name: user-auth-api
goal: Add email/password login to the Next.js app
constraints:
  - Use Prisma + Postgres (no new deps)
  - JWT sessions: 15-min access, 7-day refresh
  - All routes under /api/auth/*
acceptance:
  - POST /login → 200 + token pair on valid creds
  - POST /login → 401 on bad password (no user enumeration)
  - pnpm test passes; pnpm lint clean
```

The spec outlives any single session.

OpenSpec
https://www.youtube.com/watch?v=PQU9o_5rHC4

Extreme Layer 2: Recursive Language Models (RLM)

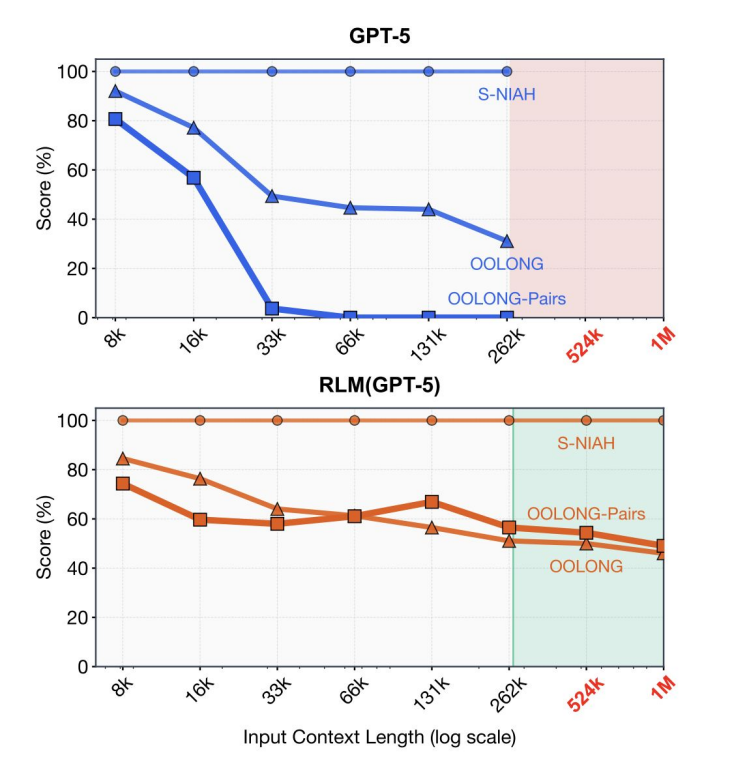
RLMs bypass token limits entirely by using a persistent Python REPL to manage their state and call sub-LLMs.



https://arxiv.org/pdf/2512.24601v1

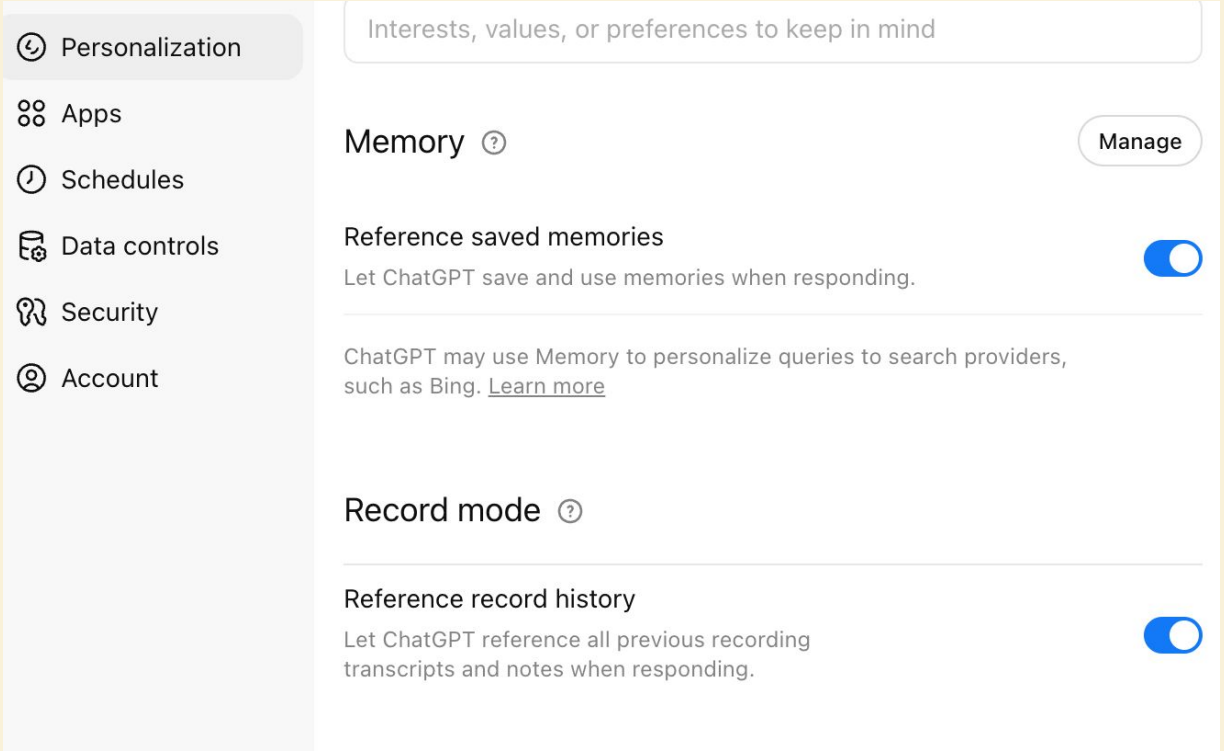
RLMs maintain accuracy at 1M tokens.

Standard GPT-5 failed entirely;
RLM-GPT-5 hit 91% accuracy



https://arxiv.org/pdf/2512.24601v1

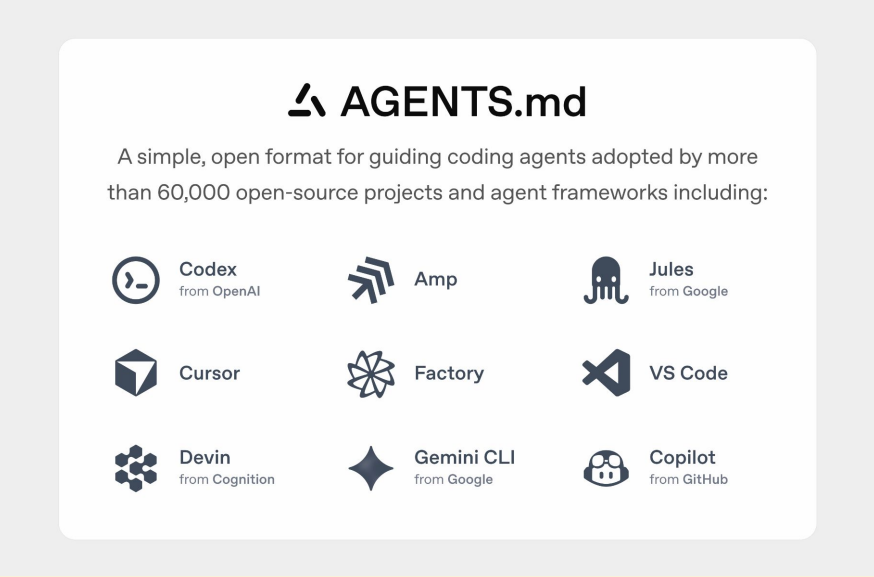
Layer 3: Durable Memory



The Reality: Unintended Distractions

Who uses an Agents.md file?

Durable Memory with Agents.md

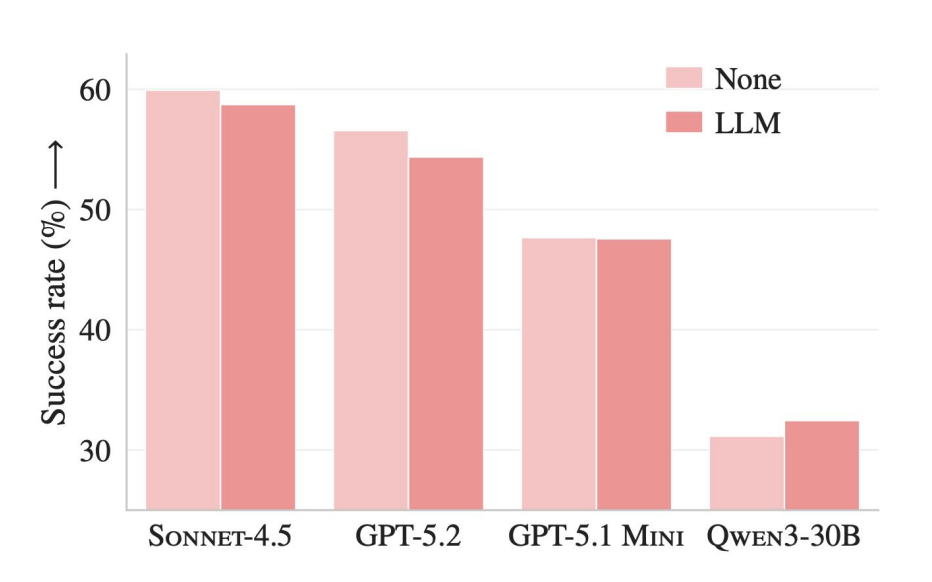



```
# Sample AGENTS.md file

## Dev environment tips
- Use `pnpm dlx turbo run where <project_name>` to jump to a package inst
- Run `pnpm install --filter <project_name>` to add the package to your v
- Use `pnpm create vite@latest <project_name> -- --template react-ts` to
- Check the name field inside each package's package.json to confirm the

## Testing instructions
- Find the CI plan in the .github/workflows folder.
- Run `pnpm turbo run test --filter <project_name>` to run every check de
- From the package root you can just call `pnpm test`. The commit should
- To focus on one step, add the Vitest pattern: `pnpm vitest run -t "<tes
- Fix any test or type errors until the whole suite is green.
- After moving files or changing imports, run `pnpm lint --filter <projec
- Add or update tests for the code you change, even if nobody asked.

## PR instructions
- Title format: [<project_name>] <Title>
- Always run `pnpm lint` and `pnpm test` before committing.
```

Don’t overload this file, it’s part of every prompt

Auto-generated AGENTS.md files hurt performance

Across multiple coding agents and LLMs, we find that context files tend to reduce task success rates compared to providing no repository context, while also increasing inference cost by over 20%.

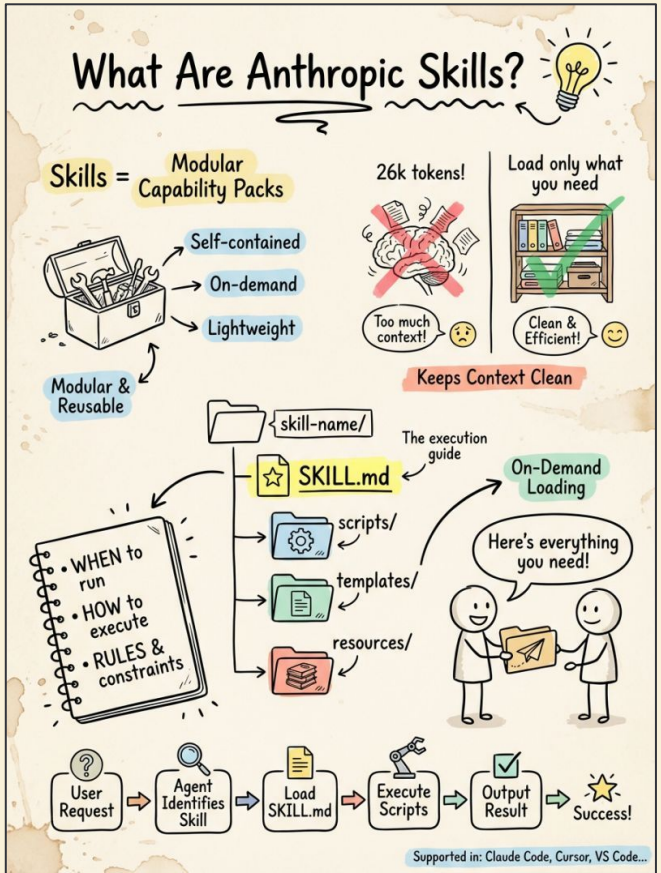


https://arxiv.org/pdf/2602.11988v1

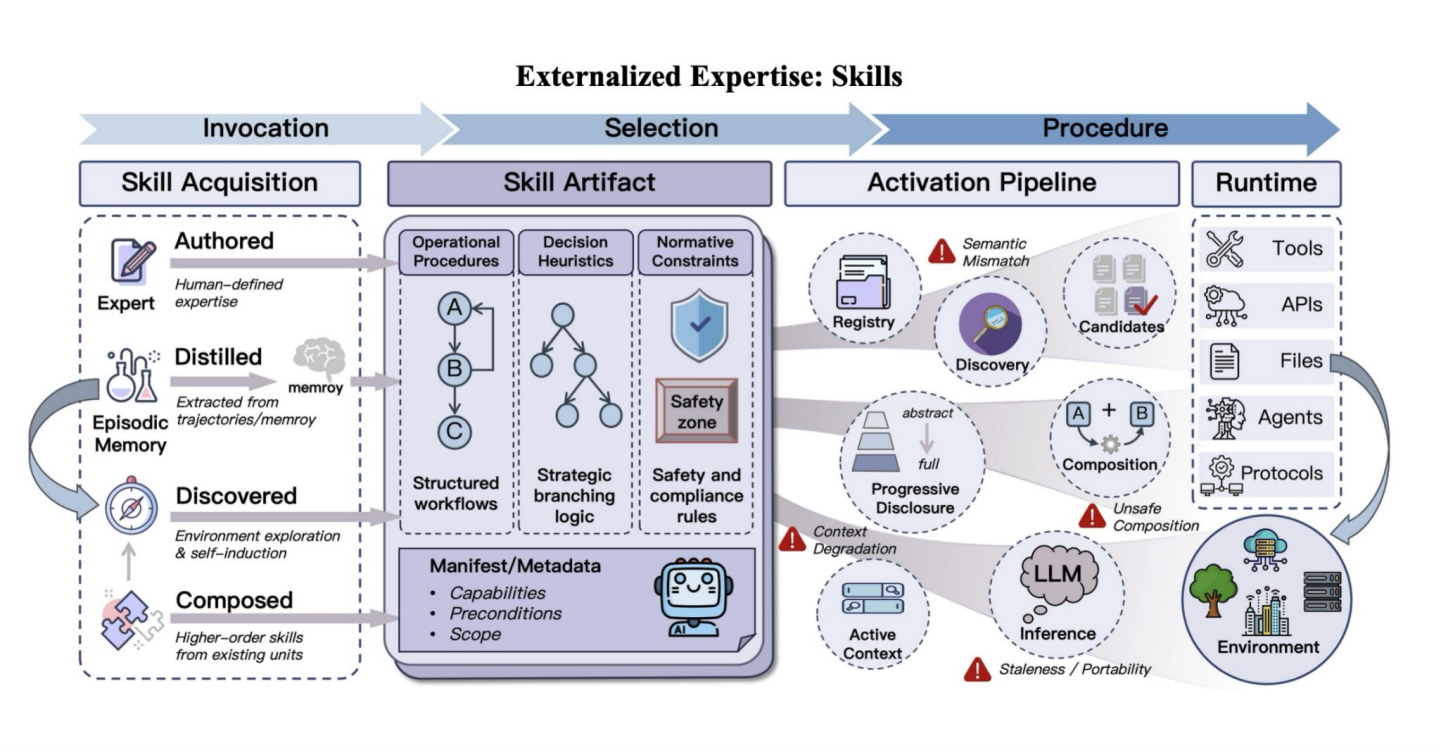
Skills are the new standard for Durable Memory.

A skill isn't just a prompt.

It's a combo of a trigger, a reference manual, and a script, e.g, a skill for querying your internal company metrics API

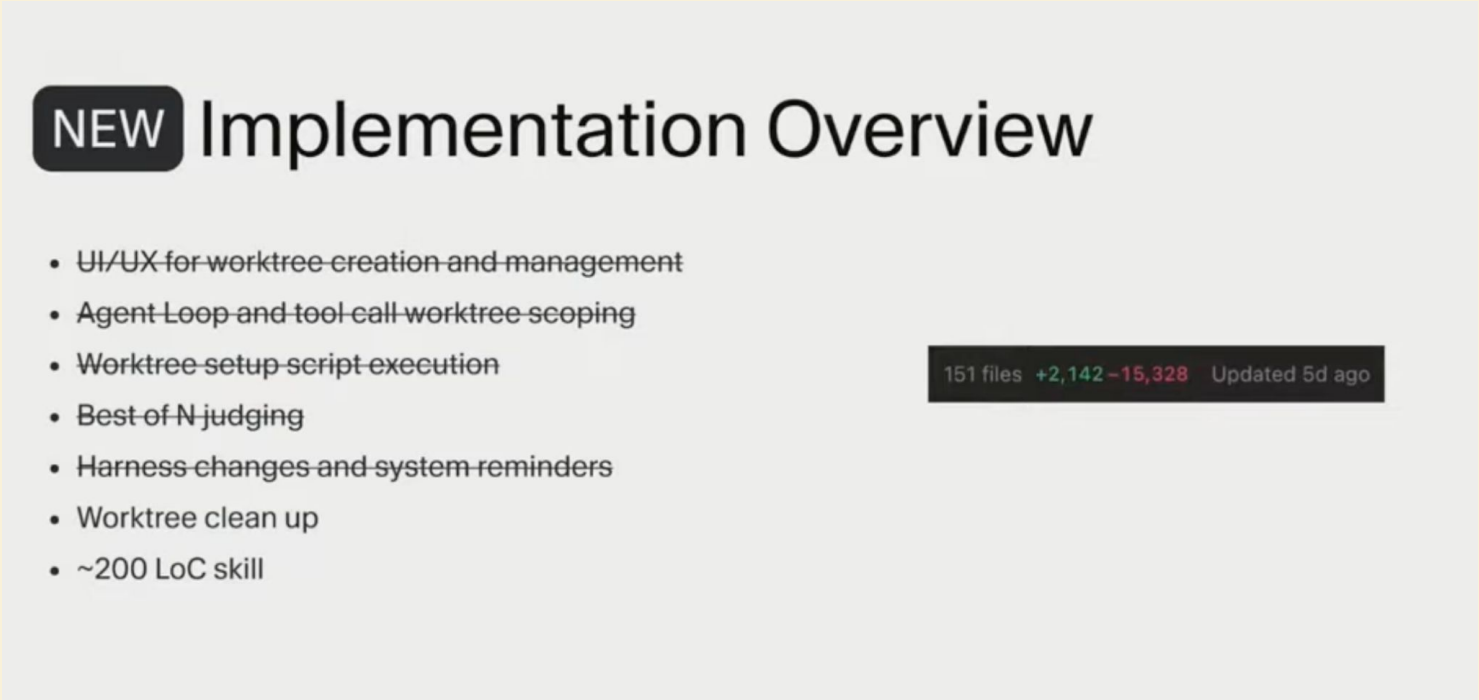


Skills as Externalized Expertise



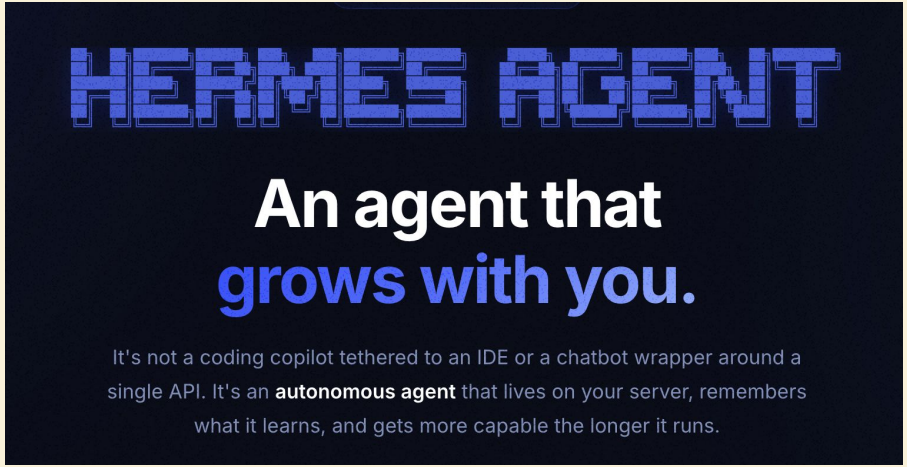
https://arxiv.org/pdf/2604.08224

Skills can replace Code



Building a learning loop

1. Task attempt: Run complex task
2. Periodic nudge: "What would you do differently?"
3. Agent writes skill: Saves to skills/.md
4. Self-improves: Edits own skill on next failure

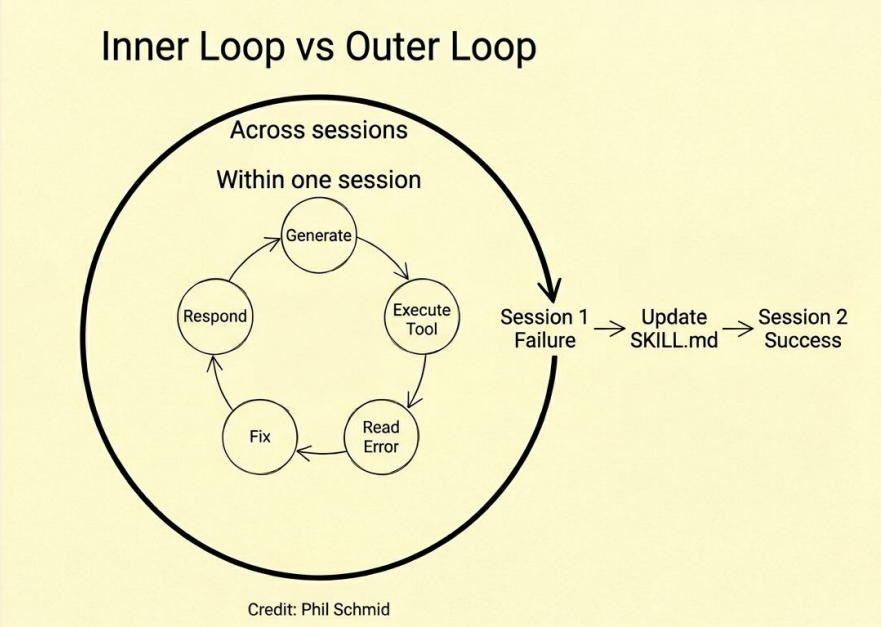


https://github.com/NousResearch/hermes-agent

Continual learning outer loop with skills

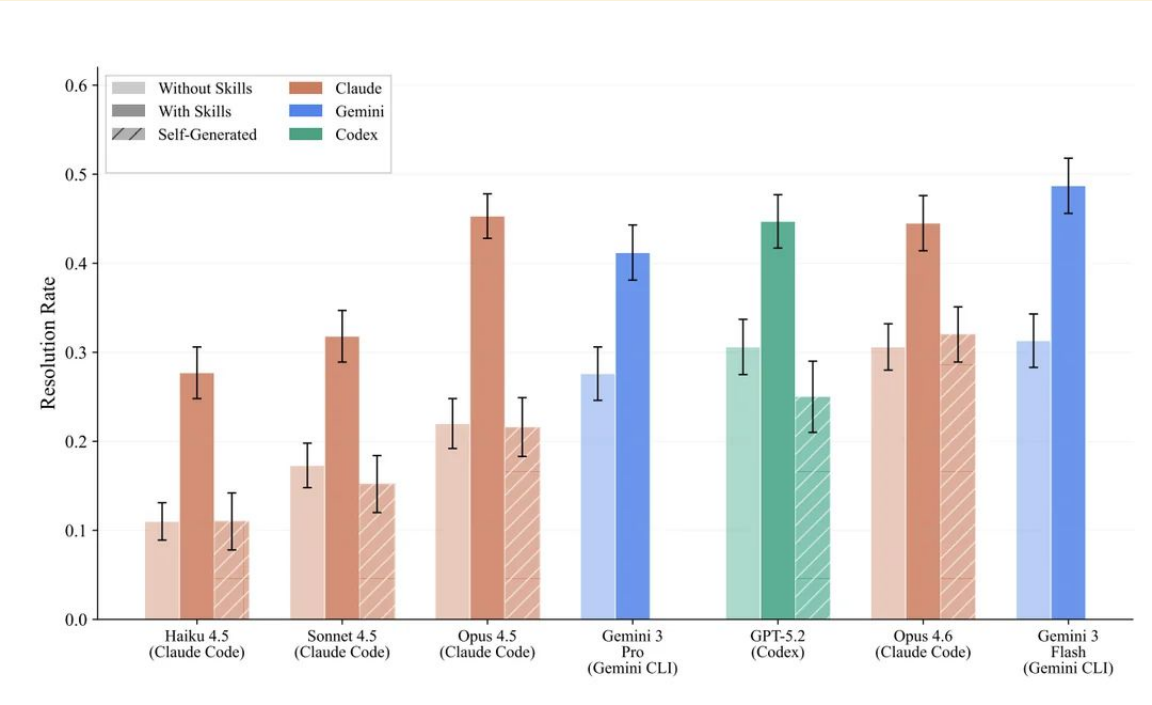
Inner loop finishes the task.

Outer loop makes the agent smarter.



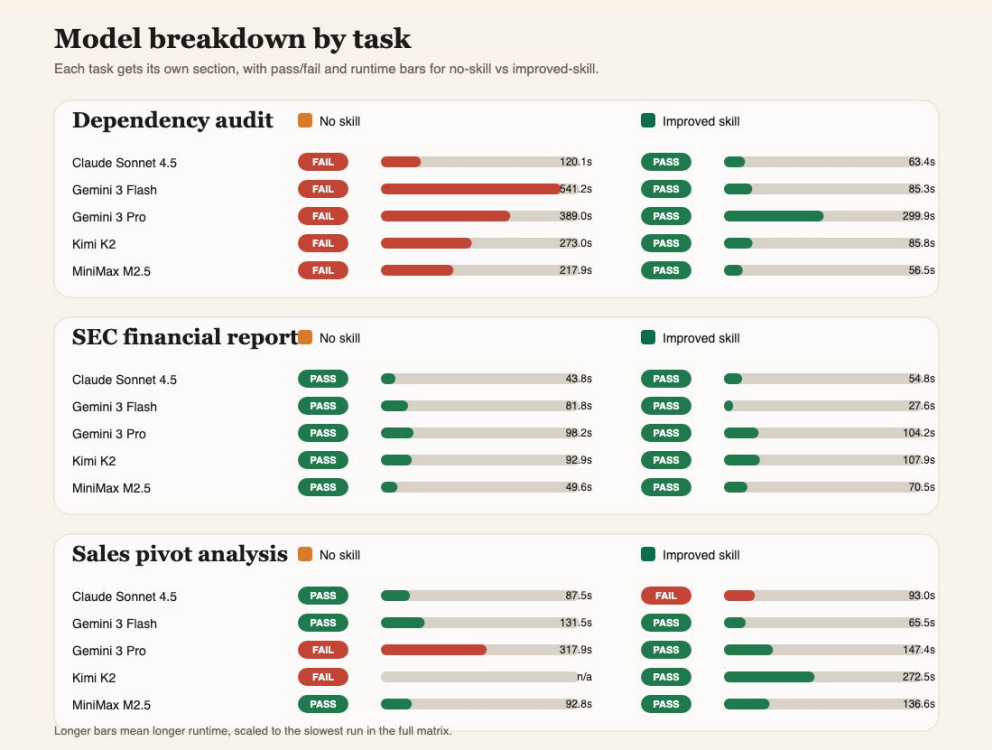
https://www.philschmid.de/inner-loop-vs-outer-loop

Warning: 16% of skills actually reduce performance.



https://www.reddit.com/r/rajistics/comments/1r77v1h/skillsbench_showed_models_arent_good_at/

You must evaluate your skills.



Blog post: https://openhands.dev/blog/evaluating-agent-skills
Repo: https://github.com/rajshah4/evaluating-skills-tutorial

The rule of thumb: "Minimize Load-Bearing Memory"

And so the thing that you want is do the minimal possible thing in order to get the model on track. And so if you delete your claude.md and then, you know, the model is getting off track, it does the wrong thing, that's when you kind of add back a little bit at a time. And what you're probably going to find is with every model you have to add less and less.

https://www.youtube.com/watch?v=PQU9o_5rHC4

Claude Code

What gets locked in:

- Conversation compaction and context assembly
- CLAUDE.md / settings / project memory
- Tool search and MCP loading strategy
- Subagent definitions and delegation behavior
- Permission rules, hooks, and policy surfaces

Memory as Lock-in

Closed Harness

- Memory lives in provider API
- Compaction is encrypted
- Switch provider → lose history

Open Harness

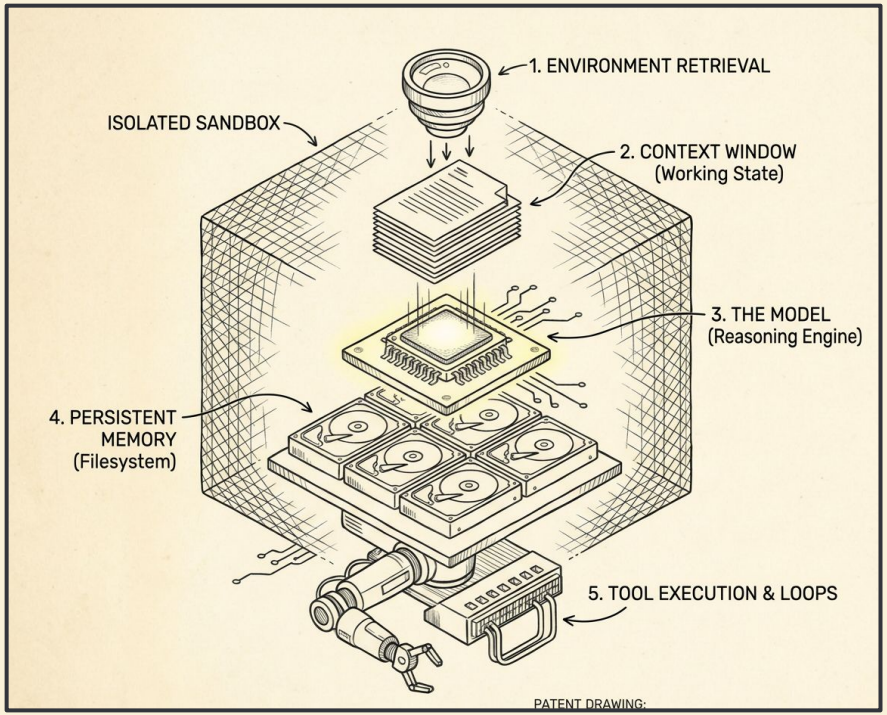
- Memory lives in your files
- Compaction is your code
- Switch provider → memory stays

Lock the harness → lose the memory → lose your product.

https://www.langchain.com/blog/your-harness-your-memory

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture



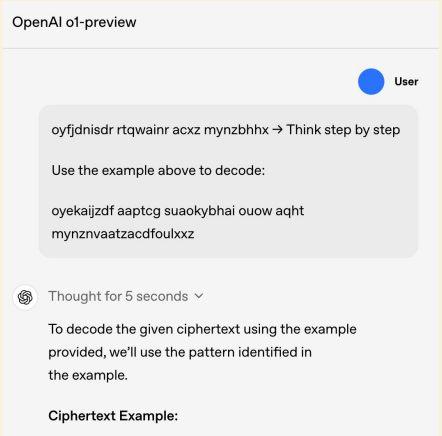
Agentic Loops and Tool Use

Better tool loops beat better prompts.

Engineering the Loop

1. The Baseline: The "Ralph Wiggum" loop (and why it fails)
2. Cognitive Discipline: Forcing the model to think via JSON schemas
3. Environmental Discipline: Using tests & CI to physically block bad loops
4. Safety & Friction: Sandboxing autonomous actions

We no longer rely on single-shot execution.



O1 - Sept 2024

https://openhands.dev/blog/openhands-context-condensation-for-more-efficient-ai-agents
https://simonwillison.net/2026/Apr/18/opus-system-prompt/

We no longer rely on single-shot execution.

Opus 4.7 Tools:

ask_user_input_v0
bash_tool
conversation_search
create_file
fetch_sports_data
image_search
message_compose_v1
places_map_display_v0
places_search
present_files
recent_chats
recipe_display_v0
recommend_claude_apps
search_mcp_registry
str_replace
suggest_connectors
view
weather_fetch
web_fetch
web_search
tool_search
visualize:read_me
visualize:show_widget

https://openhands.dev/blog/openhands-context-condensation-for-more-efficient-ai-agents
https://simonwillison.net/2026/Apr/18/opus-system-prompt/

Have you retried the same thing over and over again?

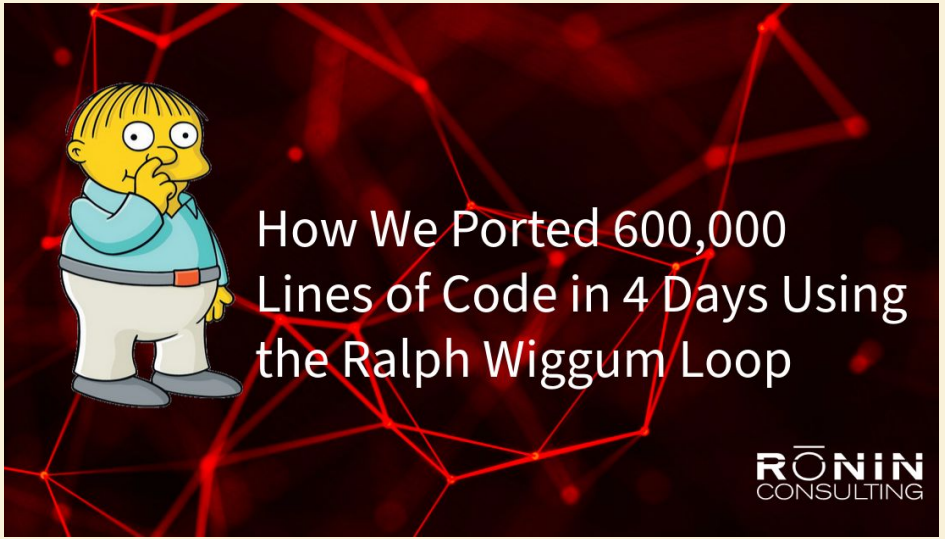
The Default: The "Ralph Wiggum" Agent

Try command
Get failure
Retry without diagnosis
Repeat until stopped by the harness

Ralph occasionally succeeds...

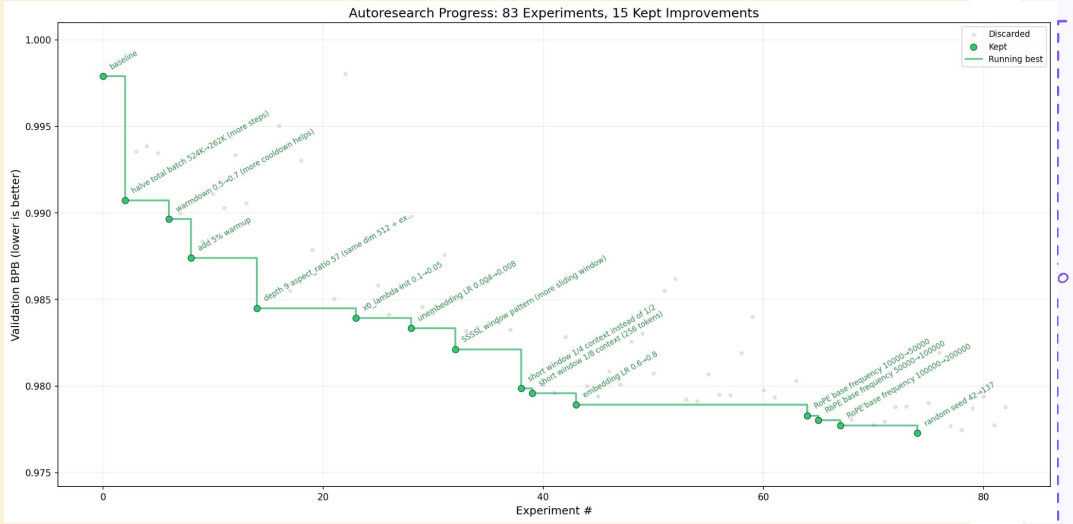
If the plan is perfect and nothing breaks, Ralph can build things.
But we can do better.

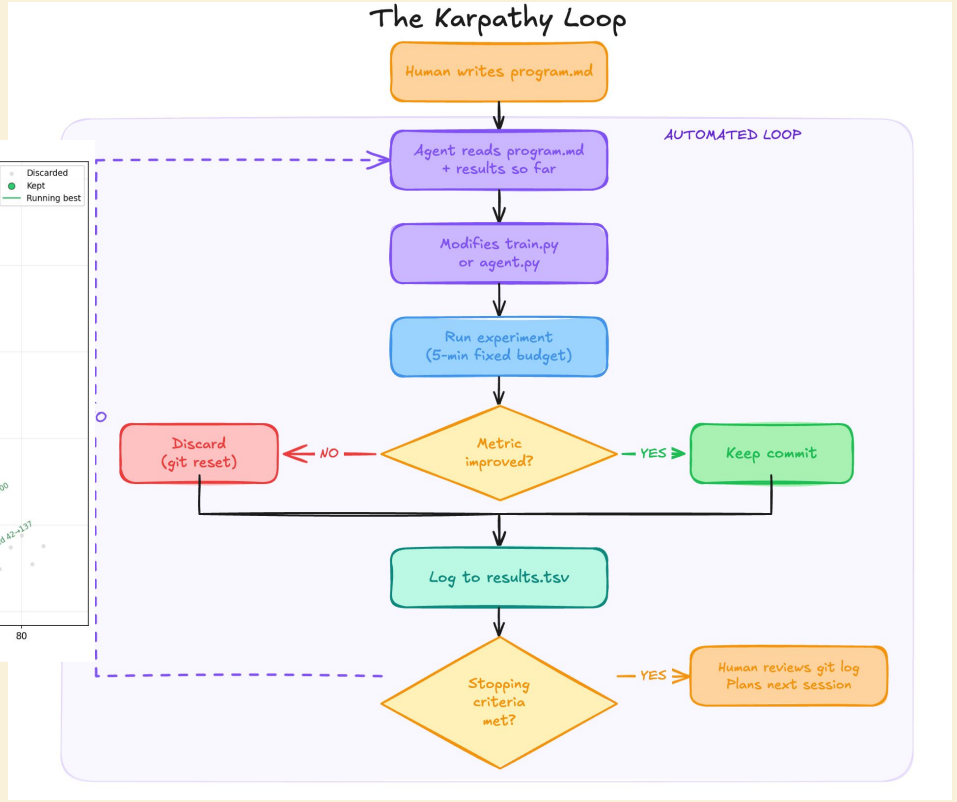
C Compiler



https://openhands.dev/blog/20260219-velocity-is-dead
https://www.ronin.consulting/artificial-intelligence/using-the-ralph-wiggum-loop/

Tool Loop for Continual Improvement

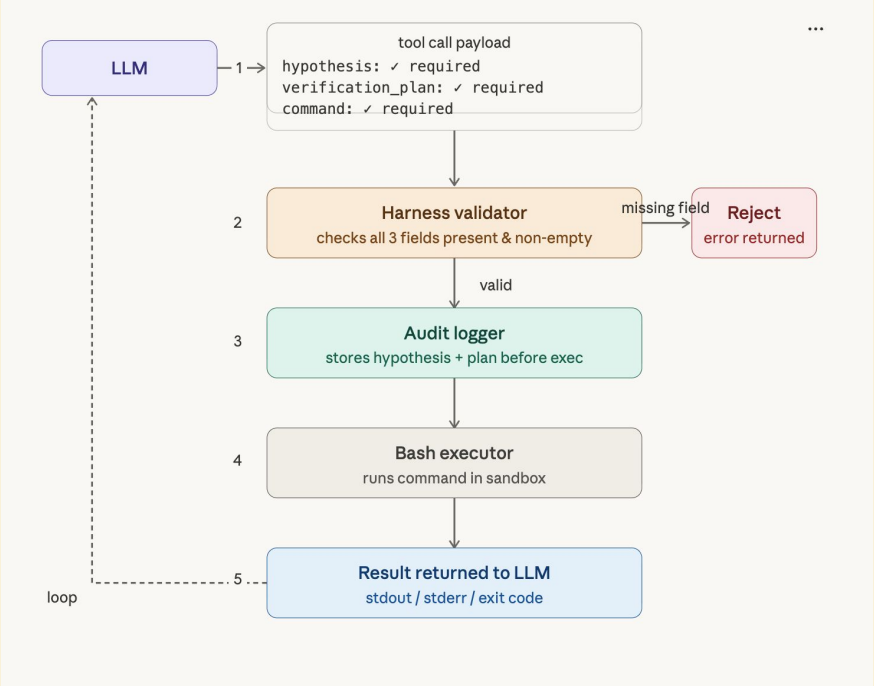




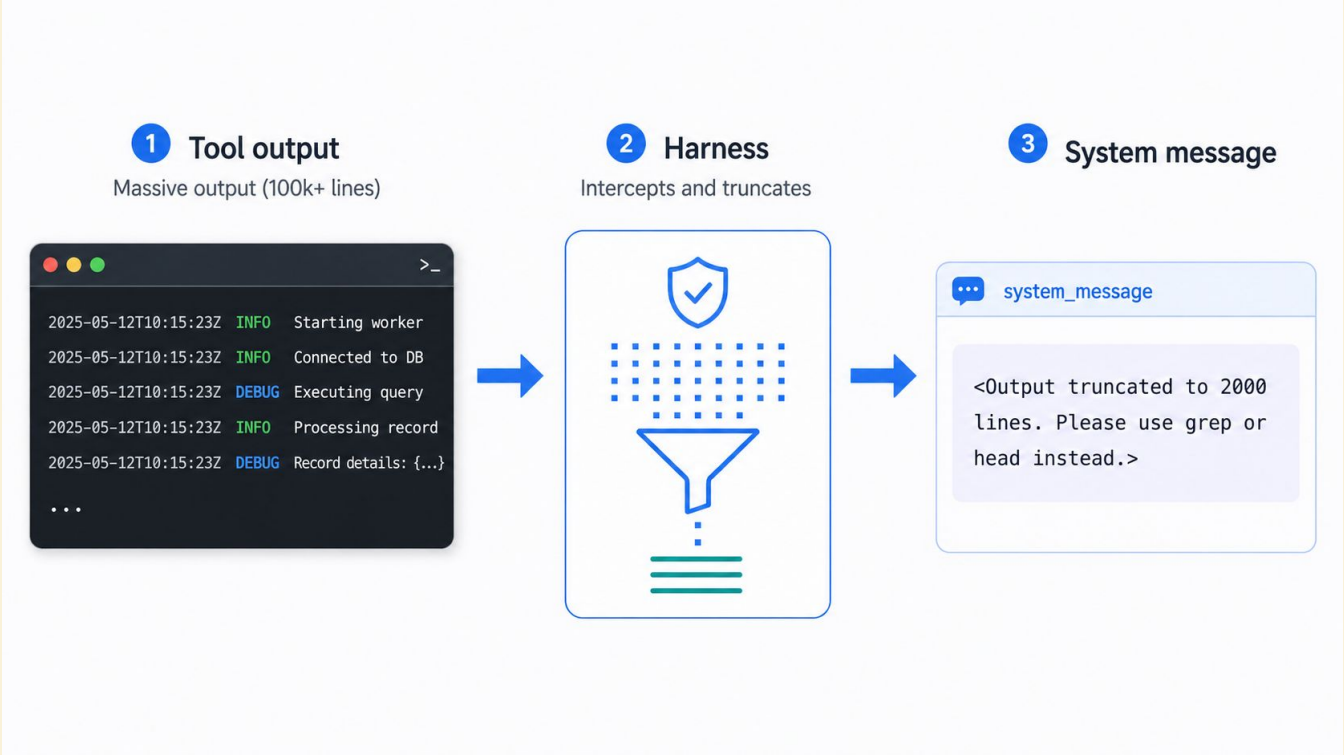
https://github.com/karpathy/autoresearch

Cognitive Discipline: Force the Hypothesis via JSON Schema.

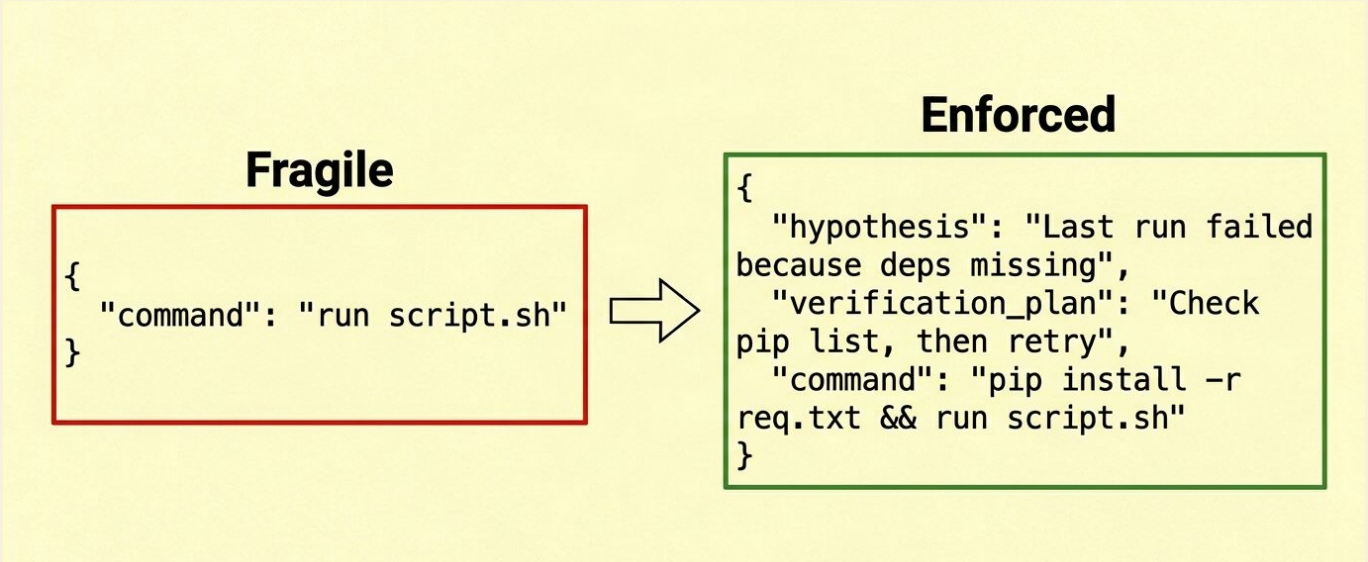
Build a plan before you execute



Defensive Tool Returns.



Externalize process



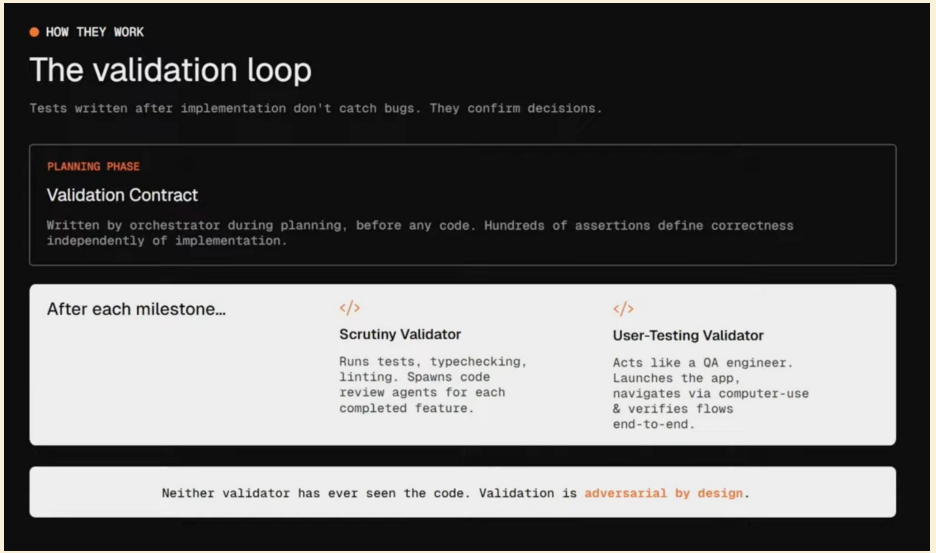
Protocols externalize the interaction burden.

https://arxiv.org/pdf/2604.08224

Environmental Discipline: Define Validation Contracts FIRST.

Tests written after implementation don't catch bugs; they just confirm the agent's decisions.

Enforce Validation Contracts, defining criteria before coding begins.



OpenAI's Code is Free

Code is Free. Architecture is Expensive.

Add system constraints:

🧹 Lint errors → instructions to the agent

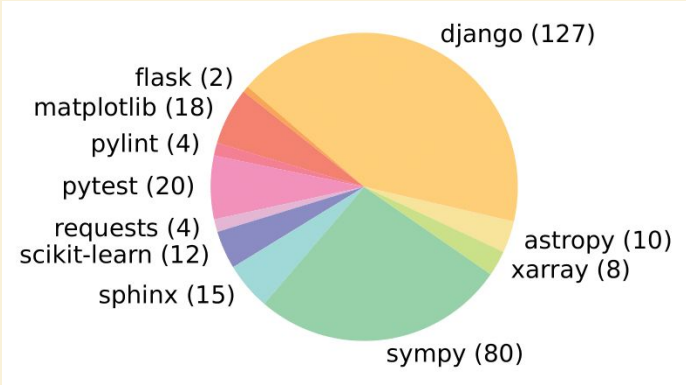
📏 File-length tests → force decomposition

🔒 CI checks → the real guardrail

https://openai.com/index/harness-engineering/

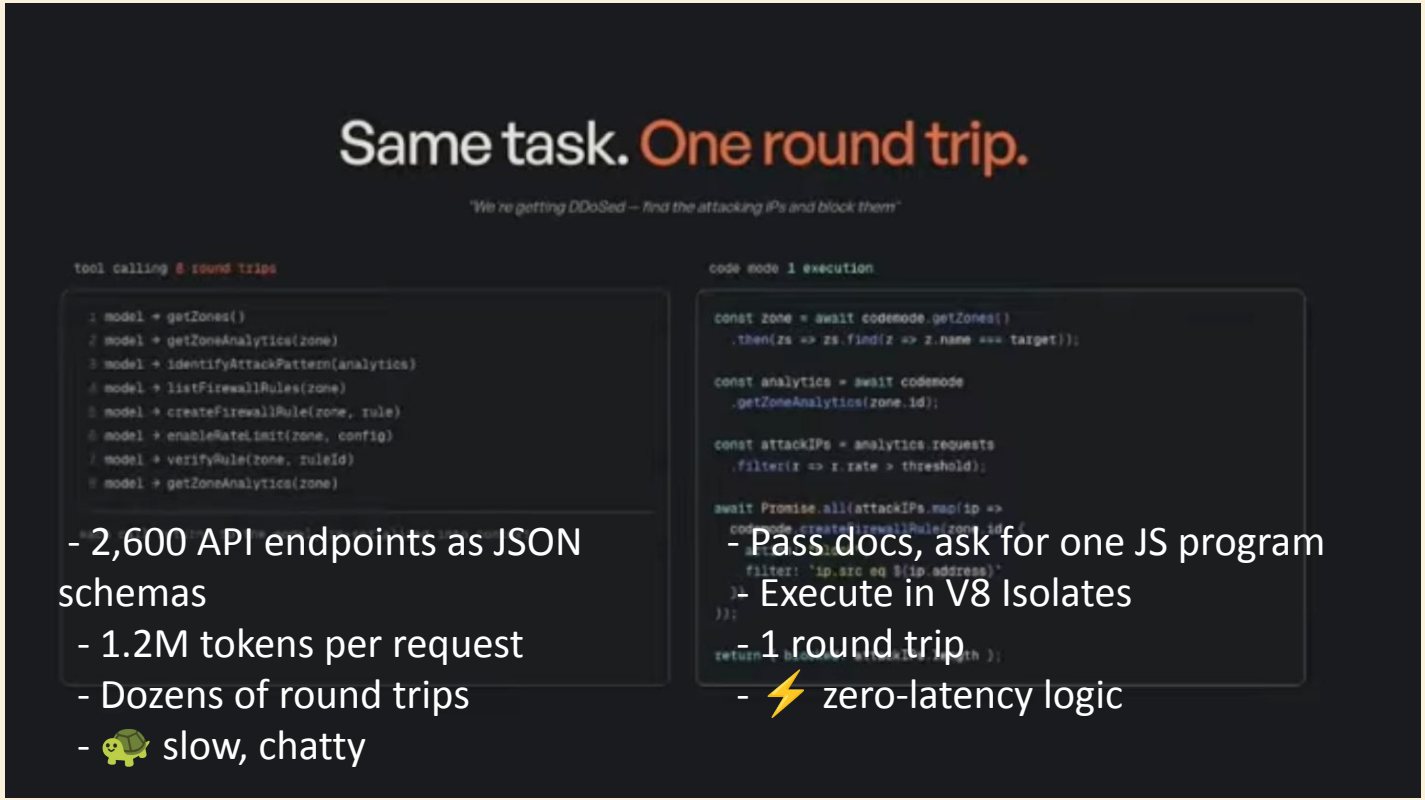
Principles for Agents

1. Actions should be simple and easy to understand for agents.
2. Actions should be compact and efficient.
3. Environment feedback should be informative but concise.
4. Guardrails mitigate error propagation and hasten recovery



SWE Bench: https://proceedings.neurips.cc/paper_files/paper/2024/file/5a7c947568c1b1328ccc5230172e1e7c-Paper-Conference.pdf

From Tools to Coding



- 2,600 API endpoints as JSON schemas
- 1.2M tokens per request
- Dozens of round trips
- 🐢 slow, chatty

- Pass docs, ask for one JS program
- Execute in V8 Isolates
- 1 round trip
- ⚡ zero-latency logic

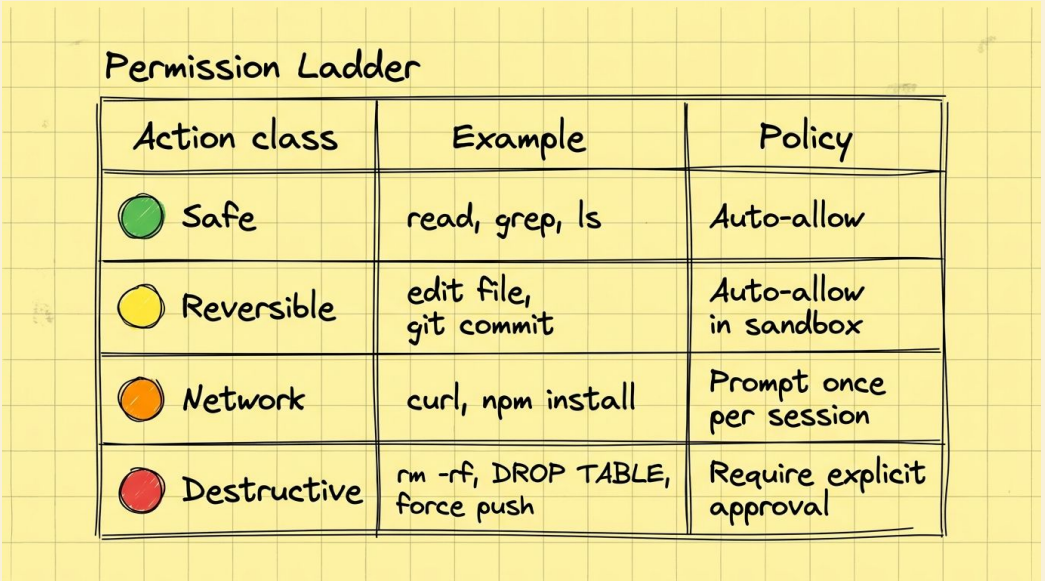
Safety & Friction: Guardrails and Approval Friction

need to add the importance
need security

https://cursor.com/blog/semsearch

Guardrails and Approval Friction

Design friction to match blast radius.



https://www.anthropic.com/engineering/beyond-permission-prompts-making-claude-code-more-secure-and-autonomous
- https://www.anthropic.com/engineering/writing-effective-tools-for-agents
- https://github.com/walkinglabs/awesome-harness-engineering

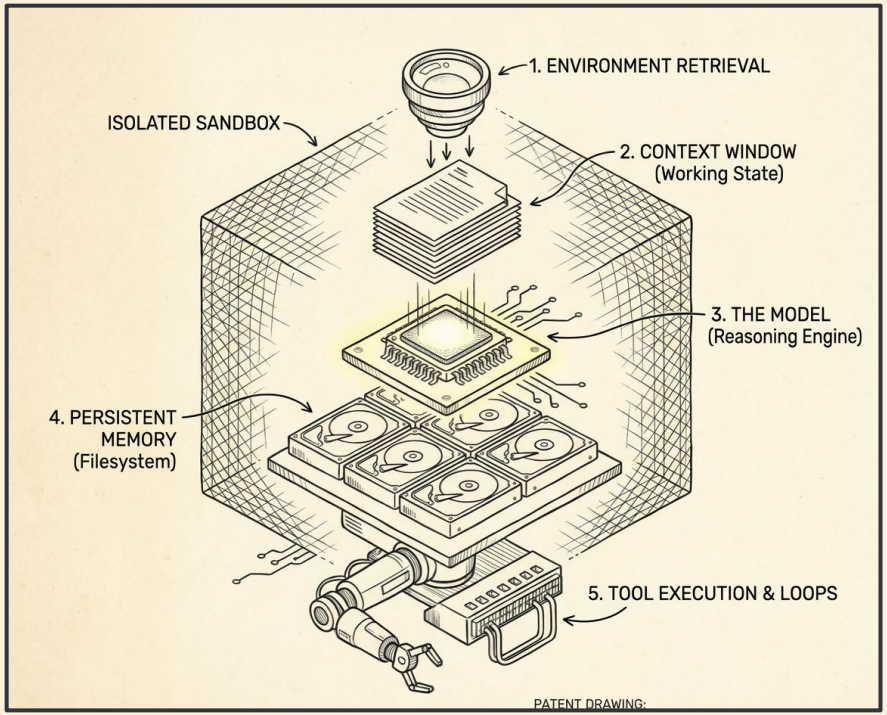
Loop Design Rules

- Make the next action explicit
- Make verification cheap
- Make blind repetition hard
- Force learning from the environment.

https://cursor.com/blog/semsearch

Let's start with how agents find what they need.

- The Model
- Retrieval
- Context & Memory
- Agentic Loops & Tool Use
- System Architecture

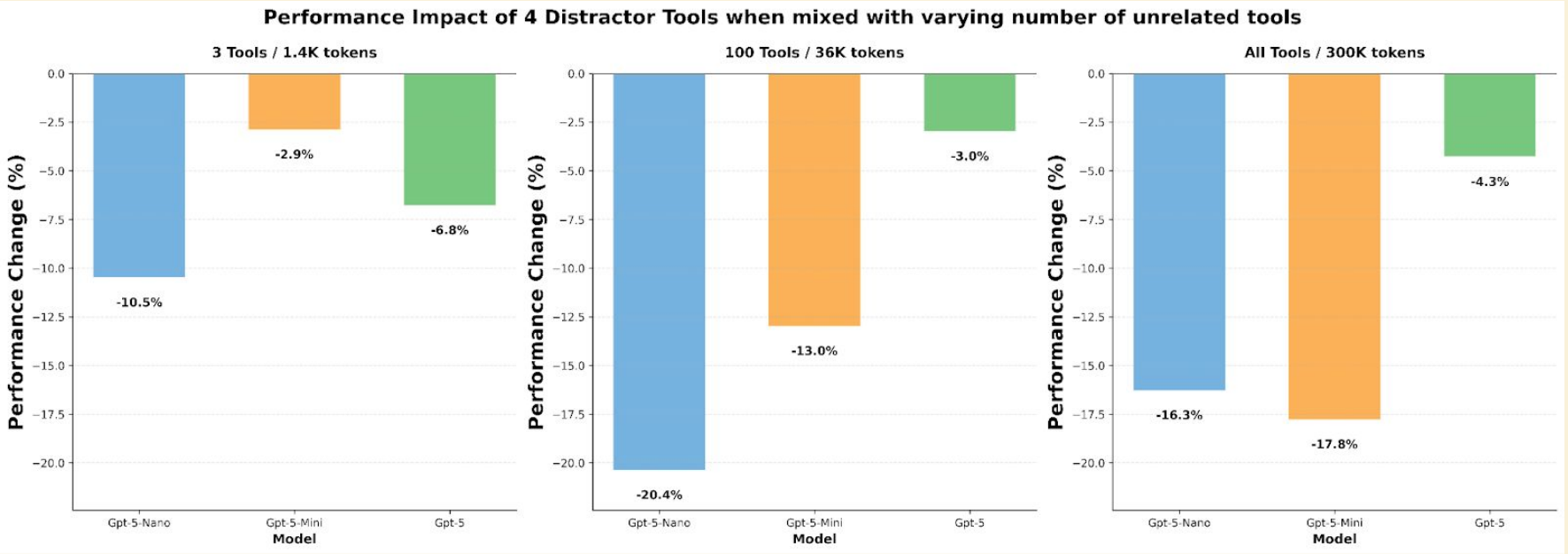


System Architecture:
Single versus Multi Agent

Architecture shapes work, state, and failures.

Who’s tried a multi-agent in production?

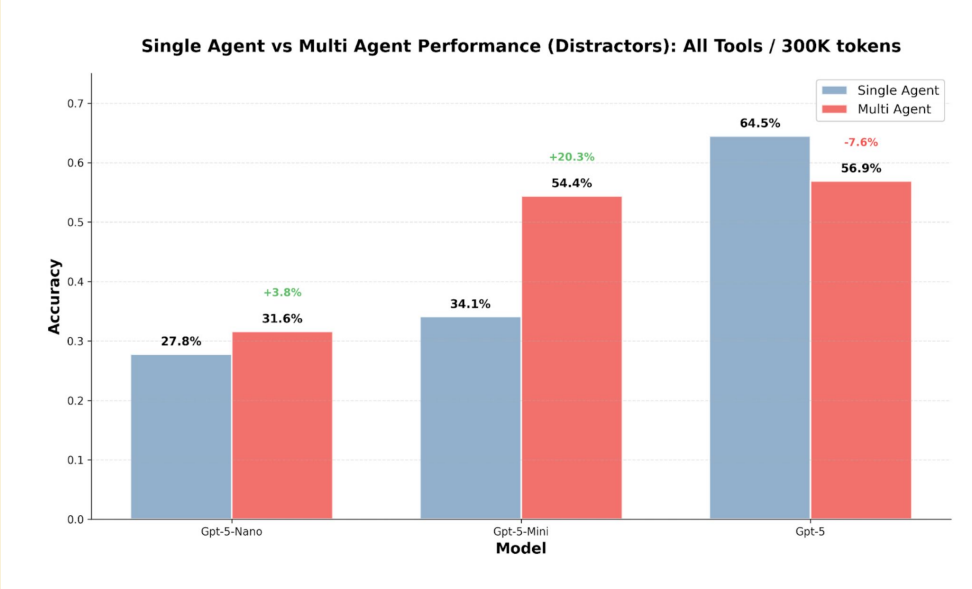
Single agents degrade as complexity grows.



As context size and the number of tools increase, even the best models start to collapse. They don't know which tool to pick."

https://snorkel.ai/blog/multi-agents-in-the-context-of-enterprise-tool-use/

The Promise: Use subagents when one agent breaks.

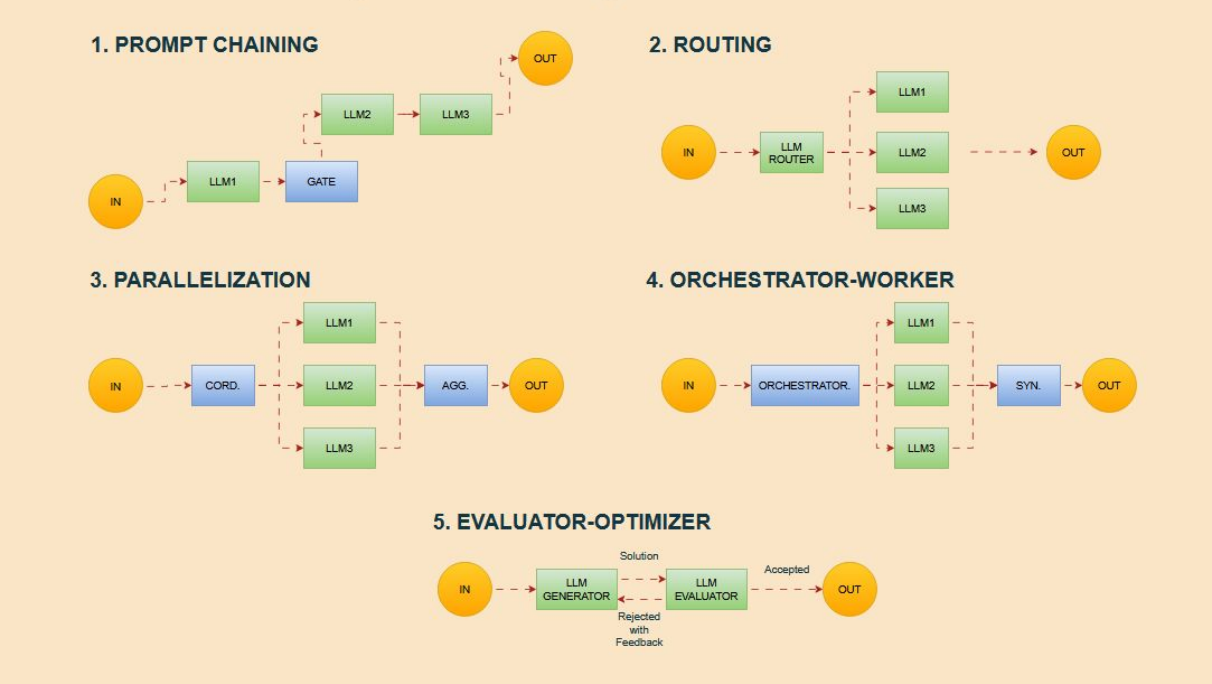


By splitting the context and tools, multi-agent systems can outperform single-agent baselines.

https://snorkel.ai/blog/multi-agents-in-the-context-of-enterprise-tool-use/

Subagents help when work splits cleanly.

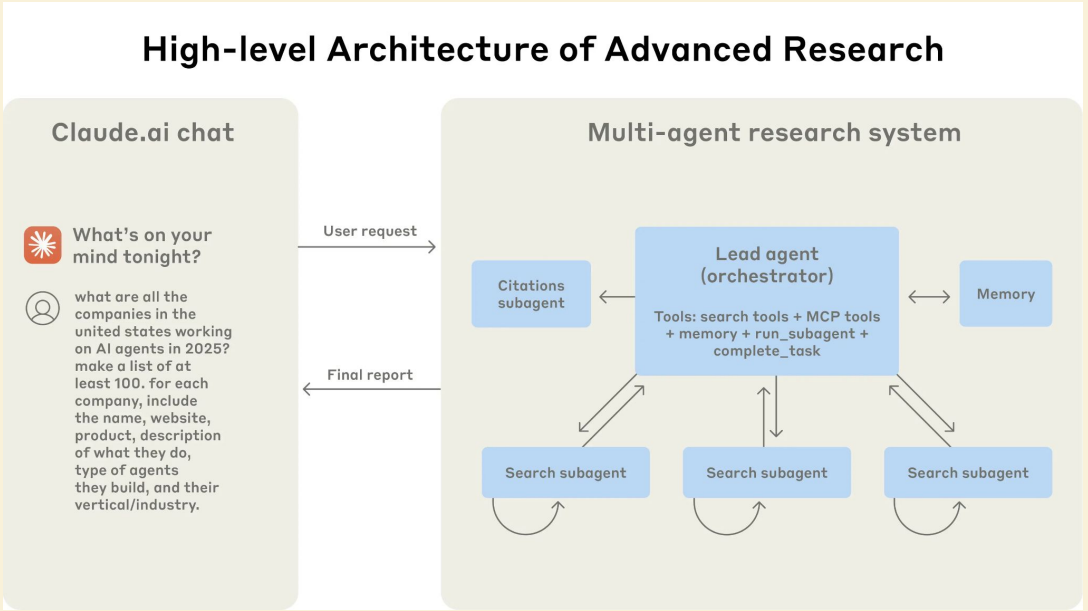
You can use basic routing, parallelization, or an Orchestrator-Worker model



https://snorkel.ai/blog/multi-agents-in-the-context-of-enterprise-tool-use/

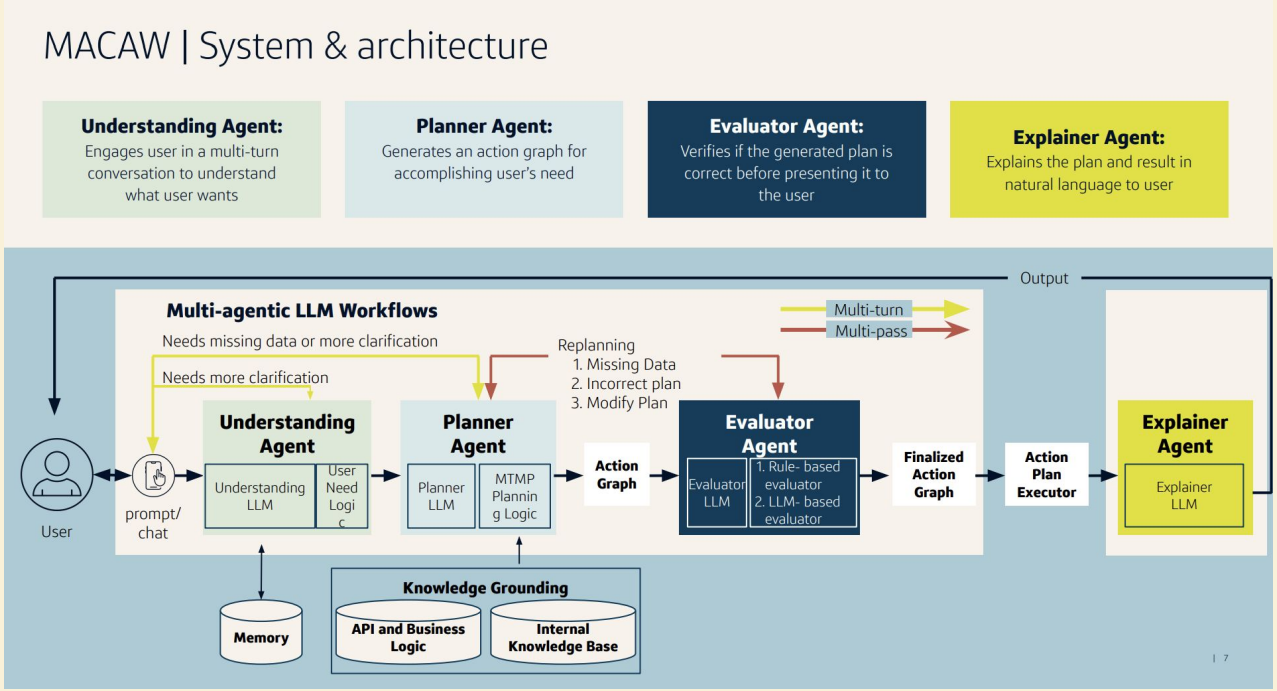
Example: Orchestrator & Workers

What are all the companies in the United States working on AI Agents in 2025?



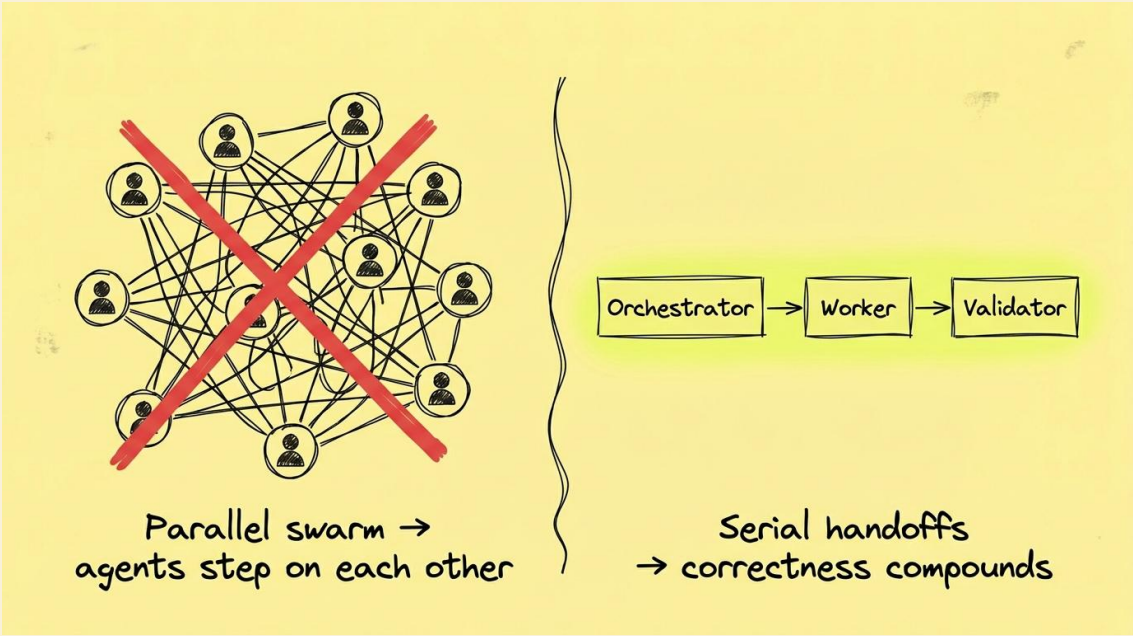
https://www.anthropic.com/engineering/multi-agent-research-system

Example: Splitting by Role



https://www.nvidia.com/en-us/on-demand/session/gtc25-s74439/

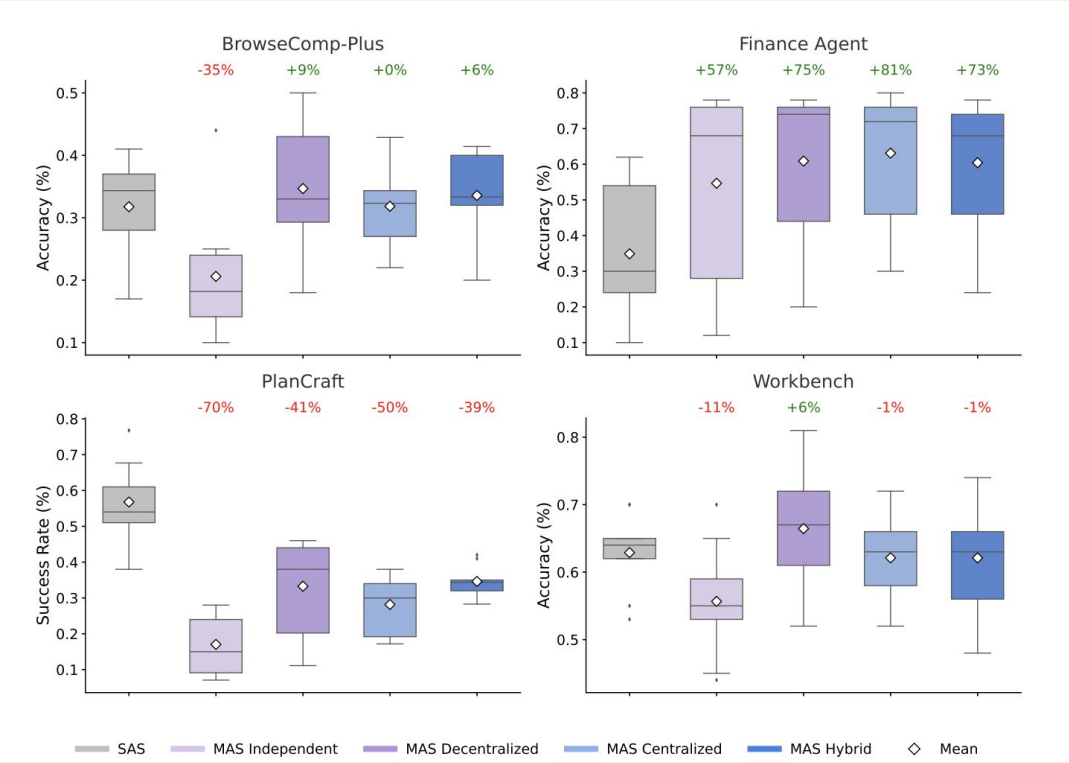
Coordination Tax - Going from Parallel to Serial



Every conflict burns tokens. Every retry erodes trust.

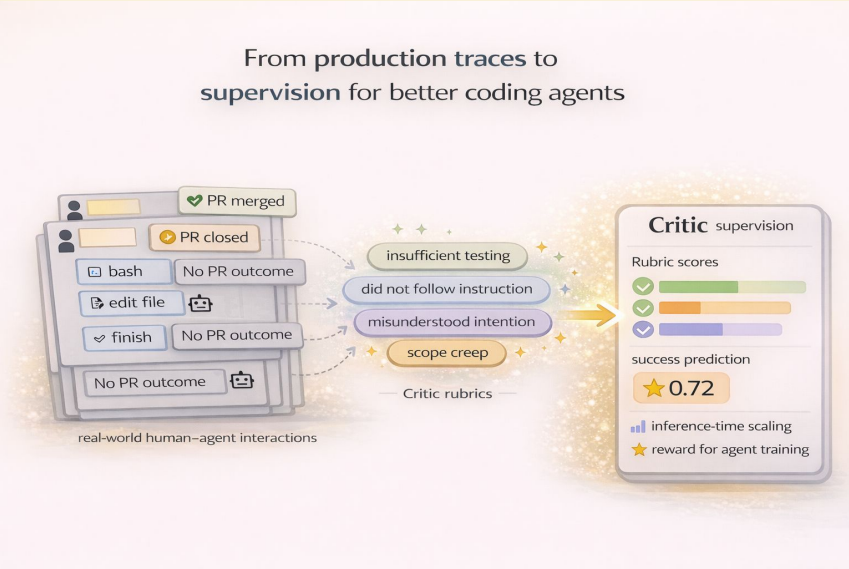
The Reality: More agents only help if coordination stays cheap.

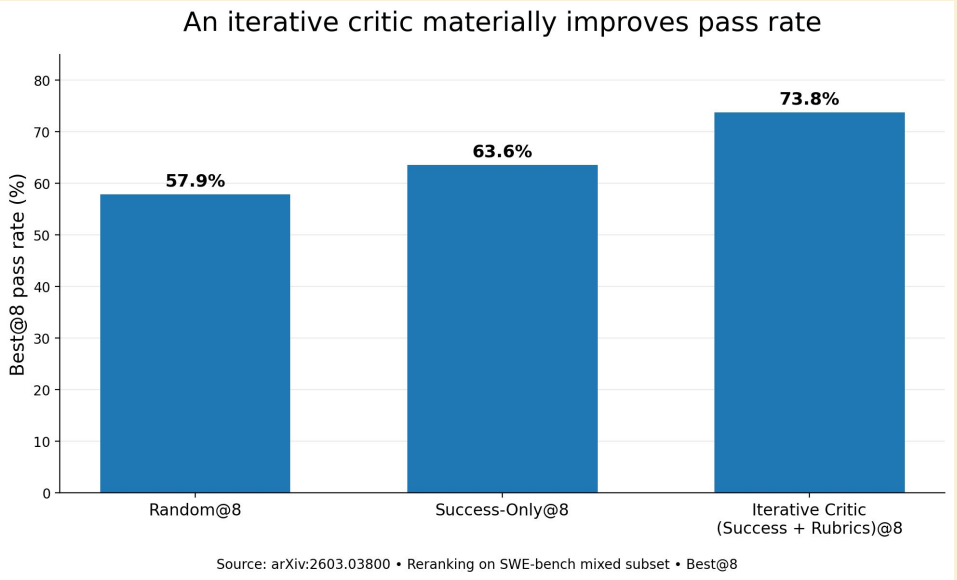
- On average, multi-agent systems perform worse than single agents (-3.5%).
- Once a single agent reaches ~45% accuracy, adding agents stops helping. Coordination cost outweighs reasoning gains.
- Architecture determines if errors are corrected or amplified. Independent agents amplify errors ~17x.



https://arxiv.org/pdf/2512.08296

Multi-Agent critics using reflection





Rubric-Supervised Critichttps://arxiv.org/pdf/2603.03800
Reflexion: https://arxiv.gg/abs/2303.11366
Boris Cherny: 2 to 3X better quality

Is harness engineering just the new prompt engineering?

Prompt engineering didn't die.

Same thing is going to happen here.

Commoditizing (will be default)	Durable (keeps mattering)
Compaction algorithms	Skills systems — your org's expertise
Default tool choices (grep, read, edit)	Memory policy — what persists, what doesn't
Model-specific prompt tweaks	Domain-specific tool libraries
Most "hacks" patching capability gaps	Security posture & approval friction
	Evals — knowing when your harness breaks

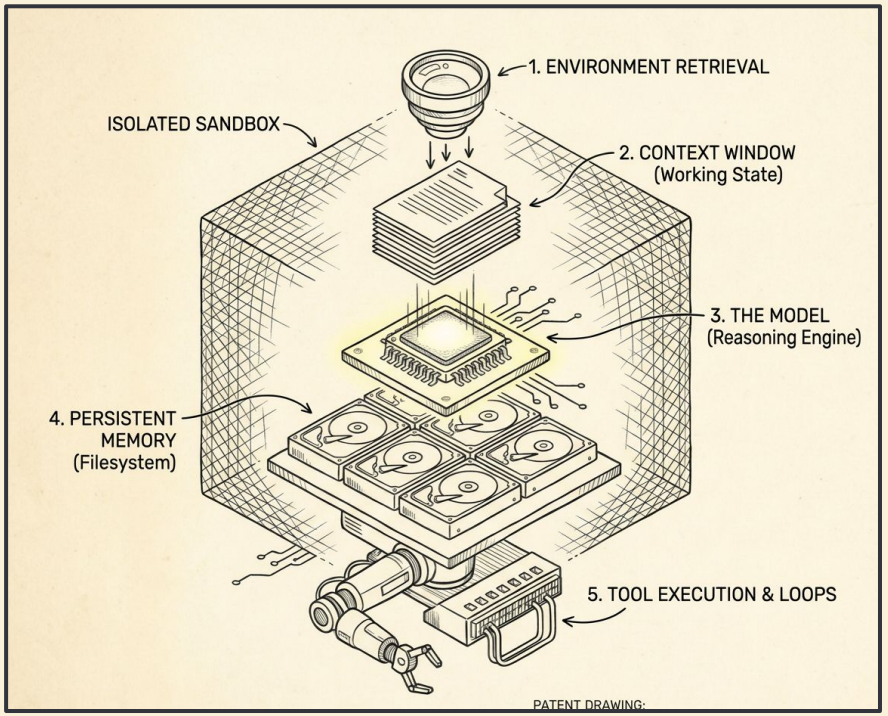
https://github.com/cobusgreyling/ai_harness_engineering

Closed or Open Harness?

1. Closed harnesses make decisions about models, memory, tools,
2. Closed harnesses are not better than open ones
3. Open ones give you flexibility to control the performance of your coding agent

Five knobs that decide everything

1. Retrieval → Grep / BM25 / semantic .
2. Memory → files > chat history. Skills
3. Tools → Quality, quantity, error handling
4. Loops → force hypothesis before action. Tests before code.
5. Orchestration → Single agent by default. Multi-agent only when coordination stays cheap.

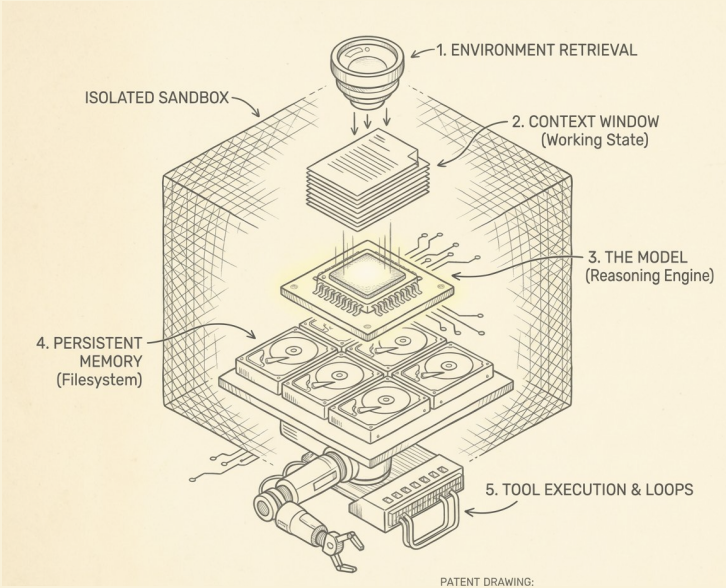


ODSC AI EAST 2026

Engineering the Harness:
A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

https://github.com/rajshah4/harness-engineering



ODSC AI EAST 2026

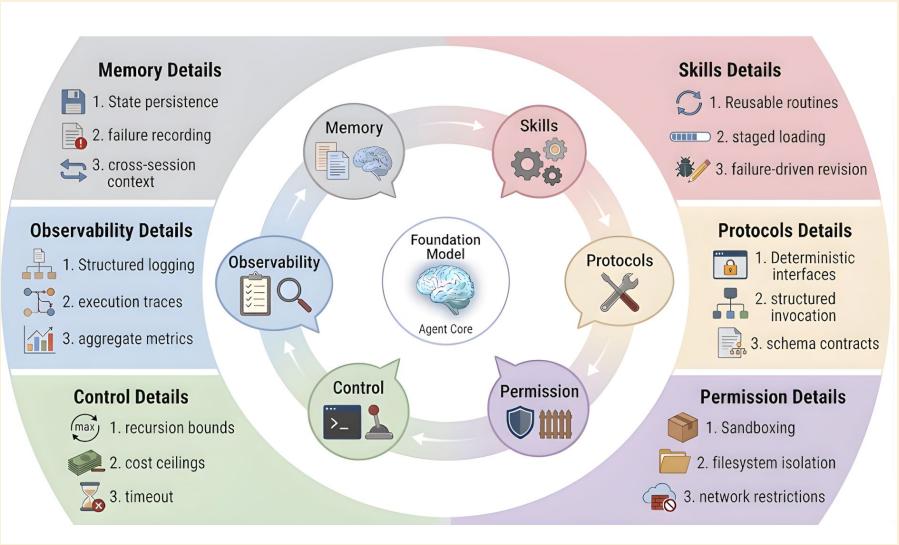
Engineering the Harness:
A Practical Workshop

Rajiv Shah
@rajistics
OpenHands

https://github.com/rajshah4/harness-engineering

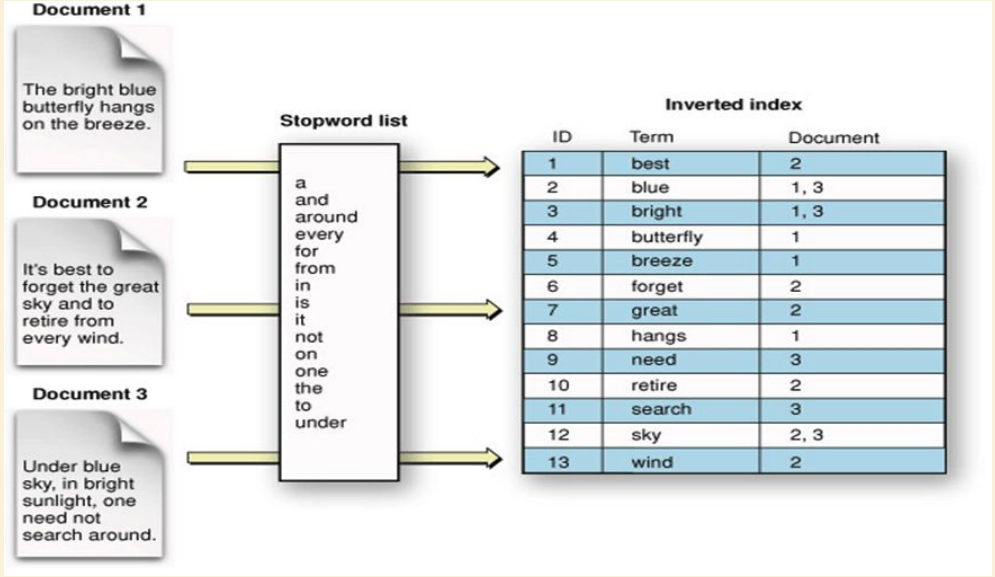
Protocols and The JSON Schema Fix

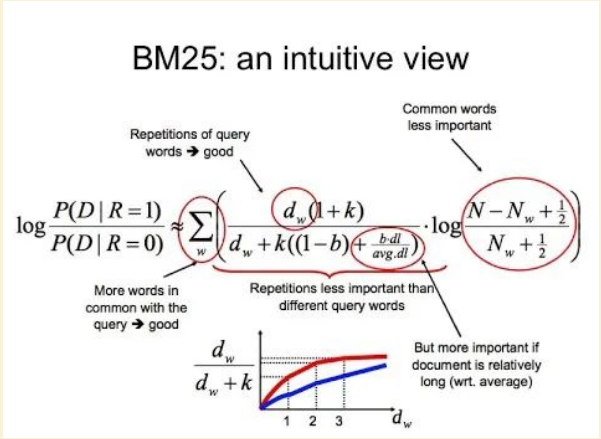
The harness as cognitive environment.
The Foundation Model (Agent Core) sits at the center.
Six harness dimensions form a coordinated ring around it.



https://arxiv.org/pdf/2604.08224

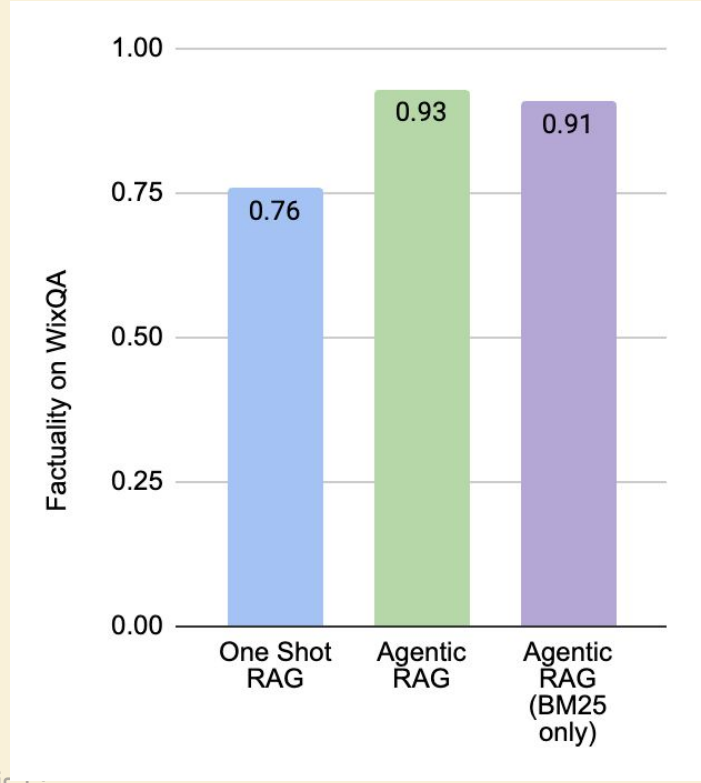
Appendix: BM25 rewards rare query terms.

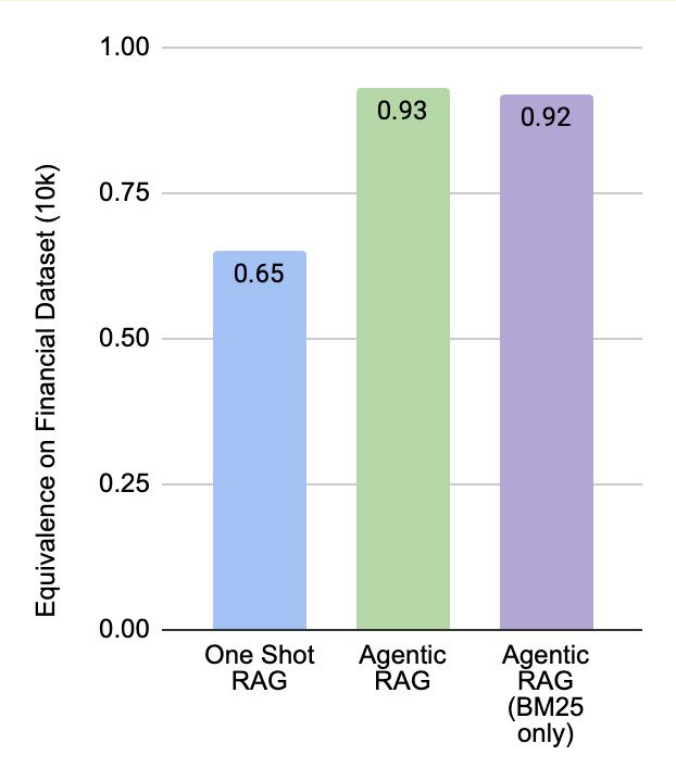




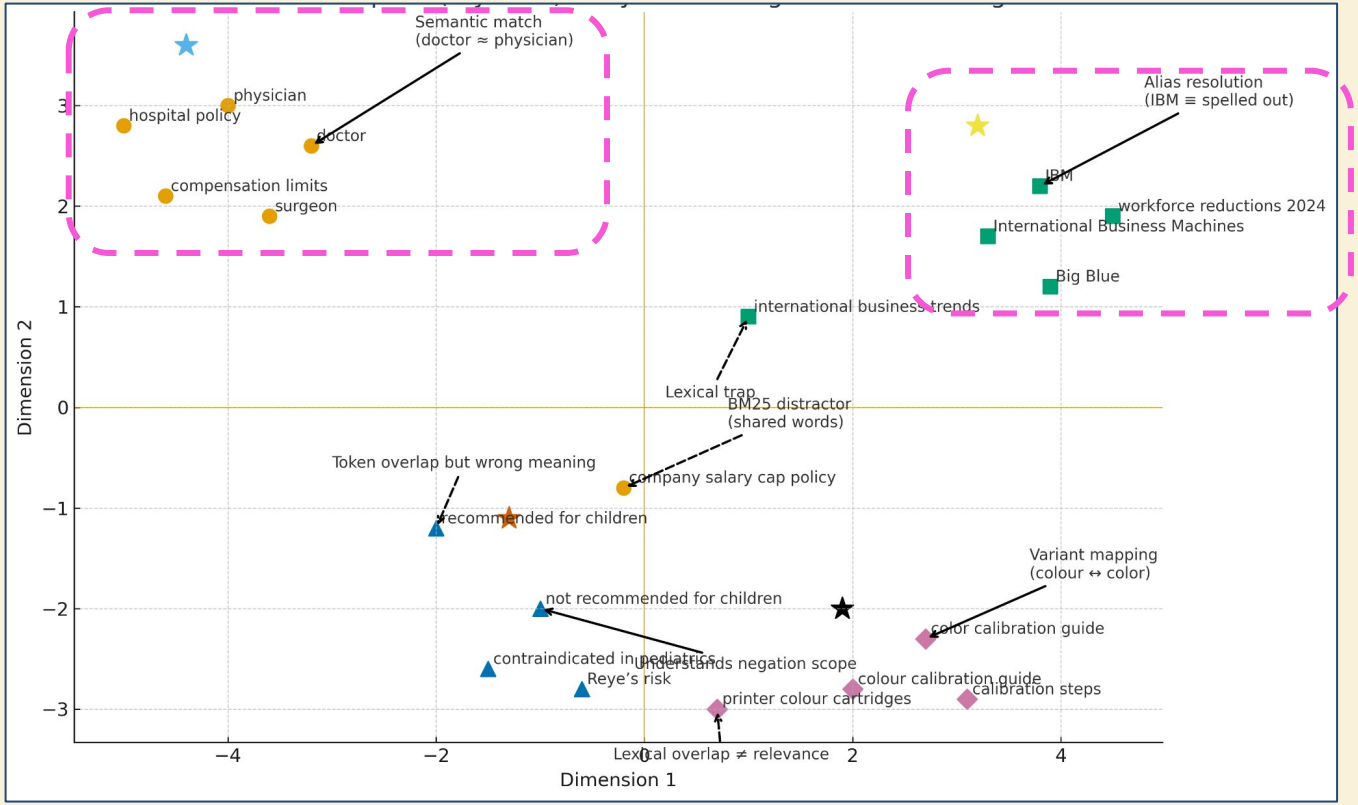
Probabilistic lexical ranking function

This holds true across domains.

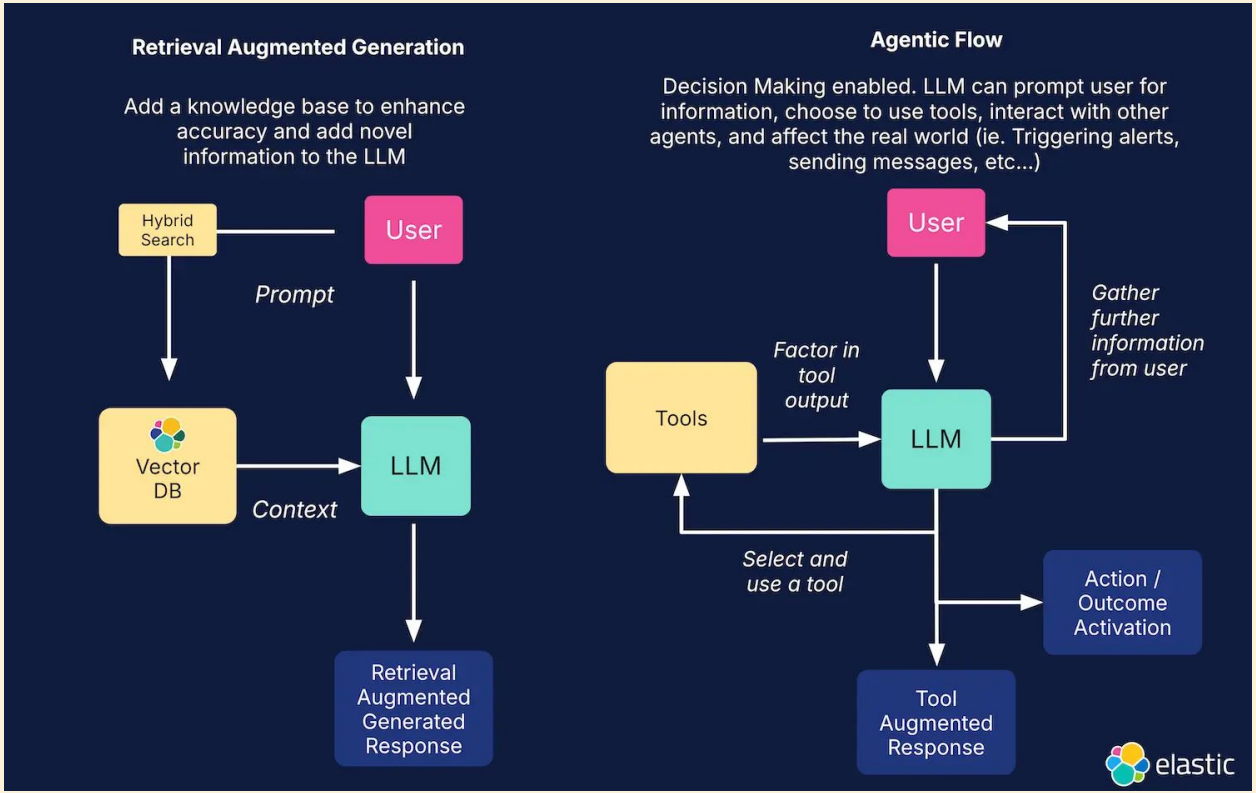




Embeddings use the semantic meaning of words

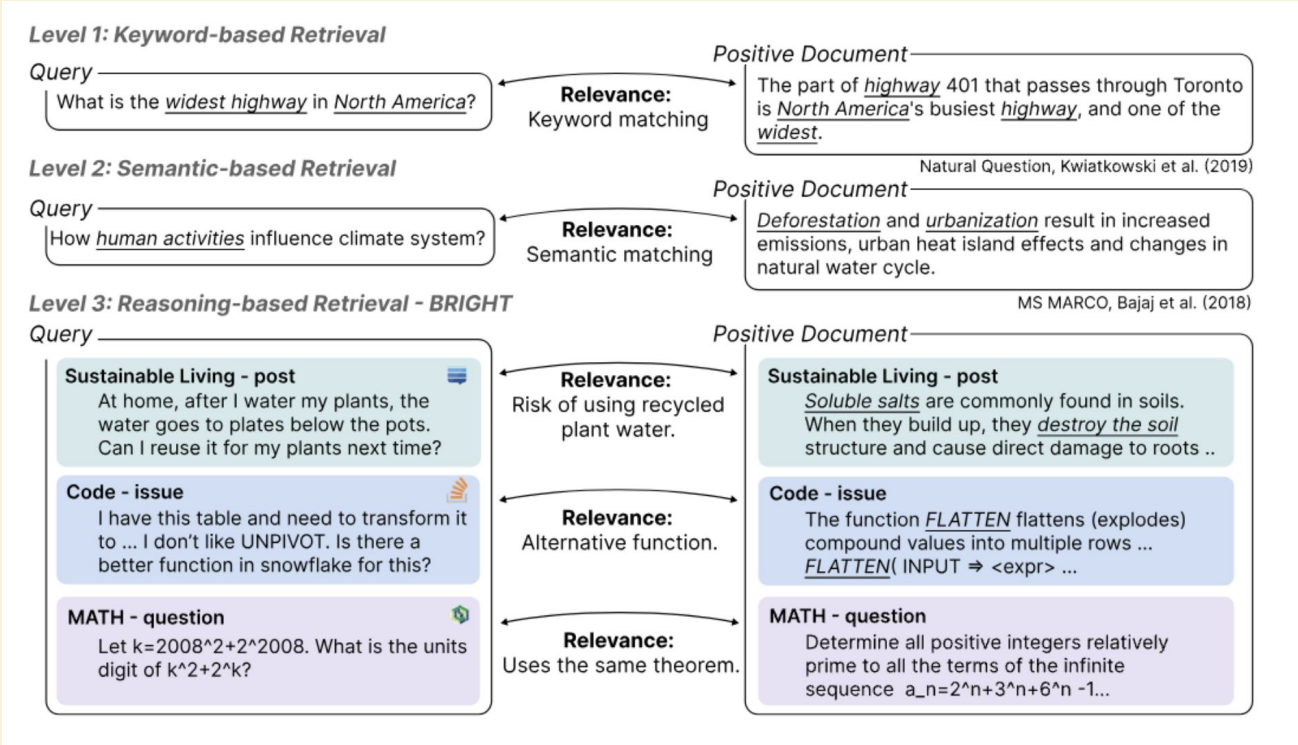


Adding a loop with agentic search



BRIGHT dataset is a more challenging retrieval task

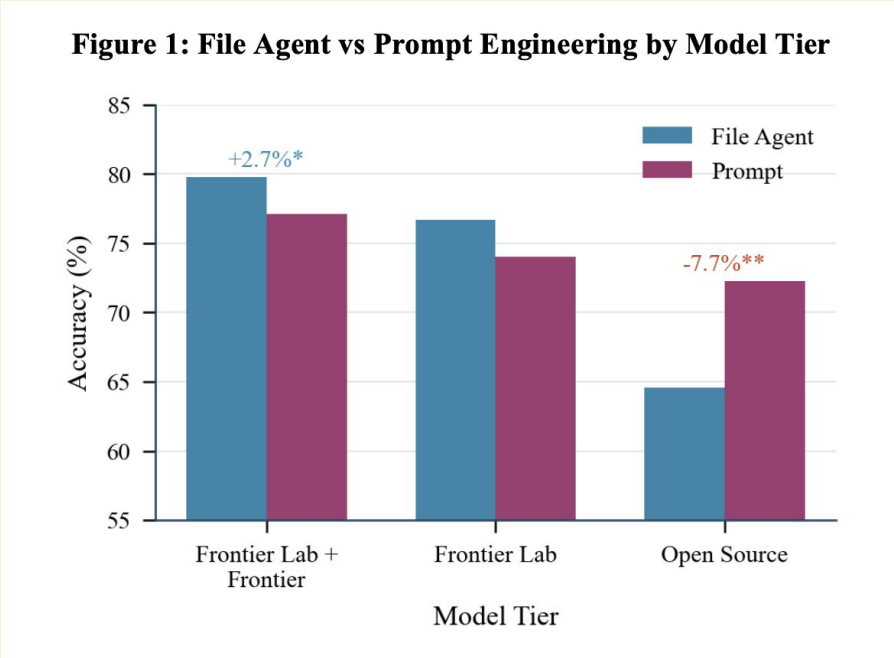
Requires reasoning over retrieval



BRIGHT: https://arxiv.org/pdf/2407.12883

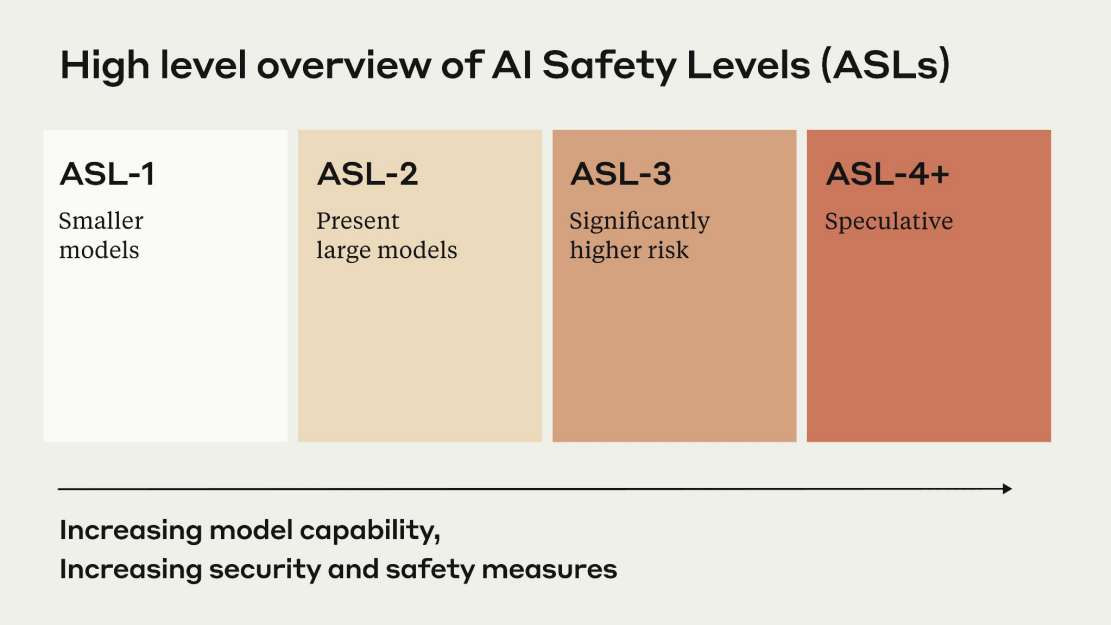
Appendix: Navigate structured data; don’t stuff into a prompt

- For structured data, file search worked better than placing schemas in prompts
- Using nested navigation guide for lots of files



https://arxiv.org/pdf/2602.05447

Appendix: Harnesses may eventually self-improve.



Models can learn frm themselves

Appendix: Context may eventually tune itself.

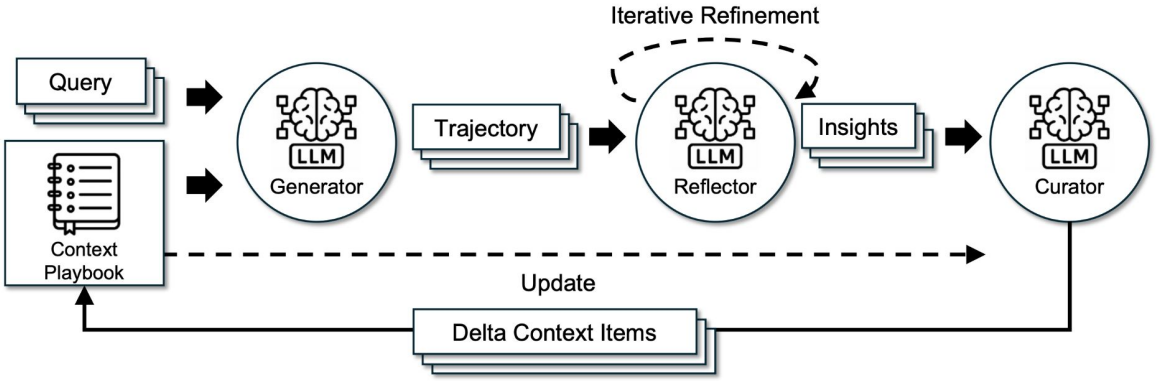


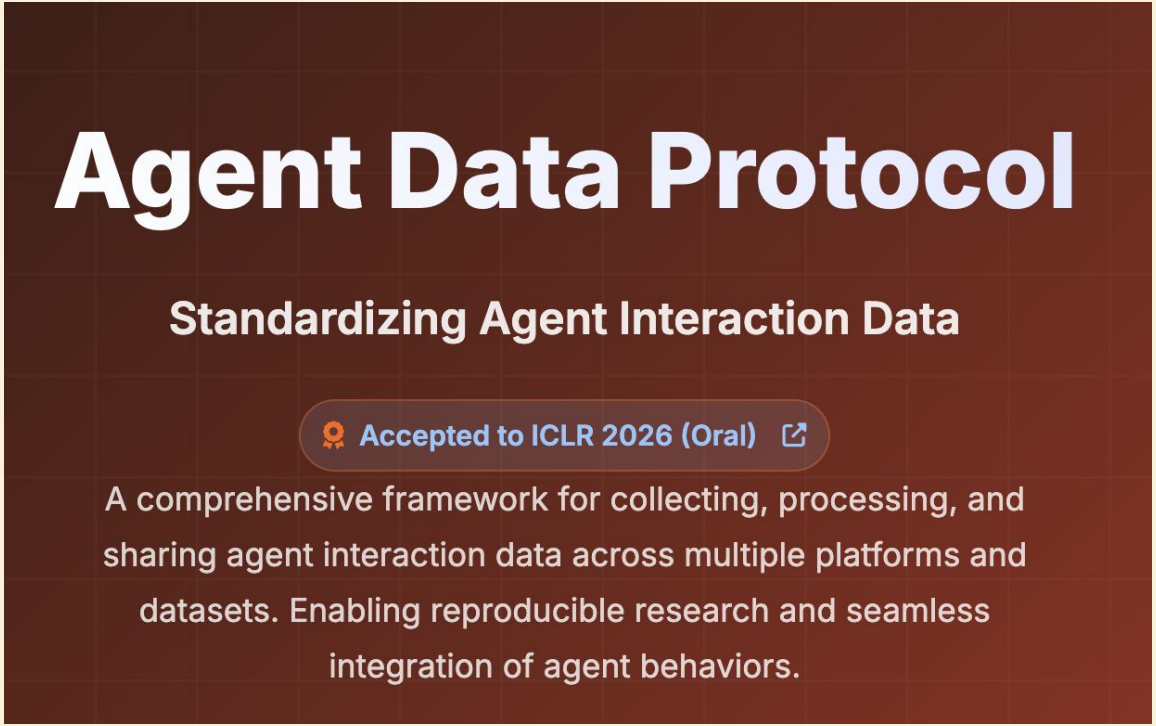
Figure 4: **The ACE Framework.** Inspired by Dynamic Cheatsheet, ACE adopts an agentic architecture with three specialized components: a Generator, a Reflector, and a Curator.

Optimizing prompts and memory

https://arxiv.org/pdf/2510.04618
https://github.com/kayba-ai/agentic-context-engine

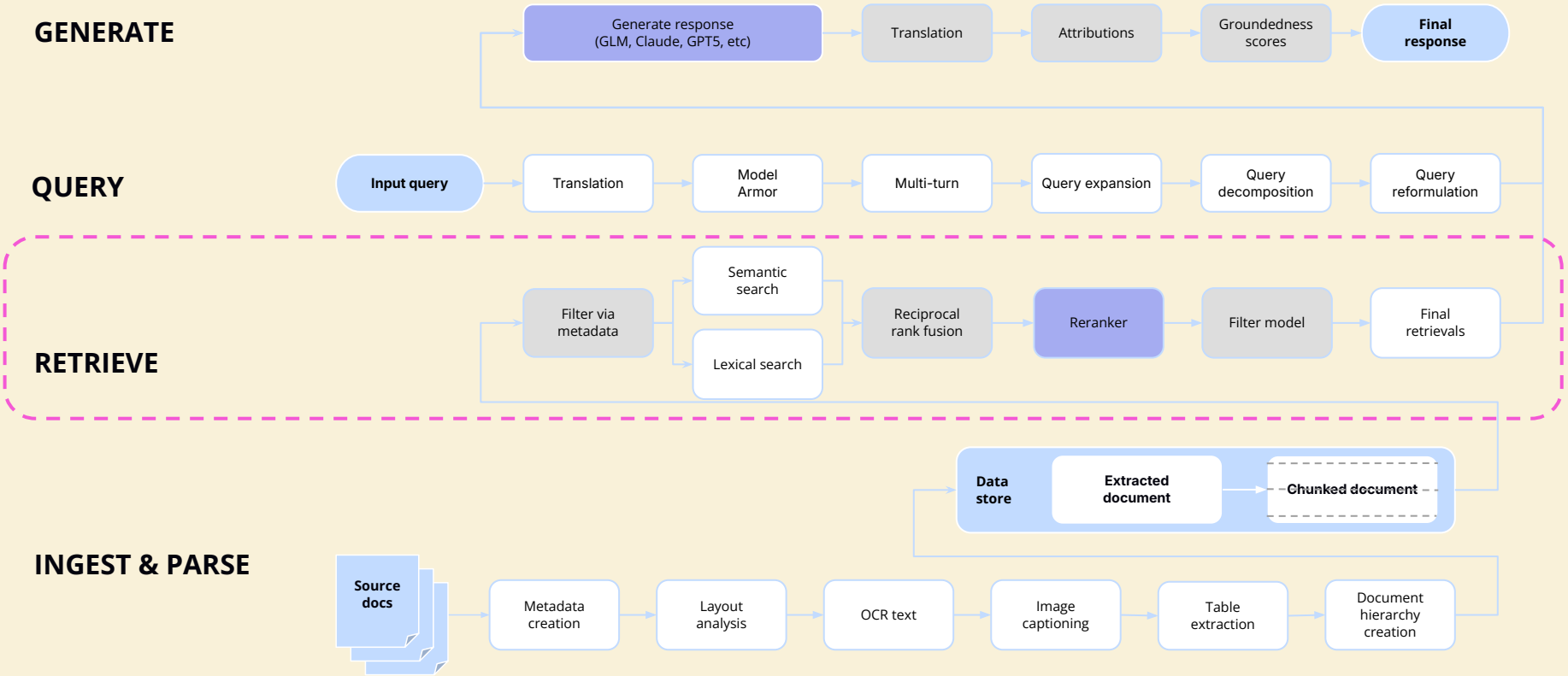
It’s possible to build an independent standard

standardized format to represent *agent trajectories across different harnesses, tools, and environments*

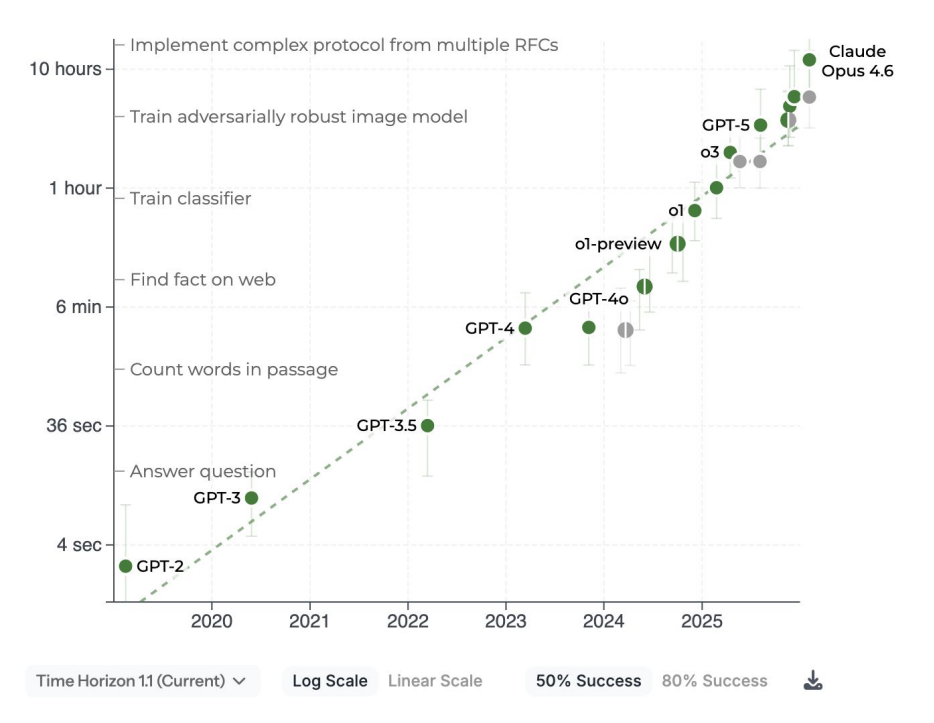


https://www.agentdataprotocol.com/

Retrieval decides what enters context.



Models are rapidly getting better.



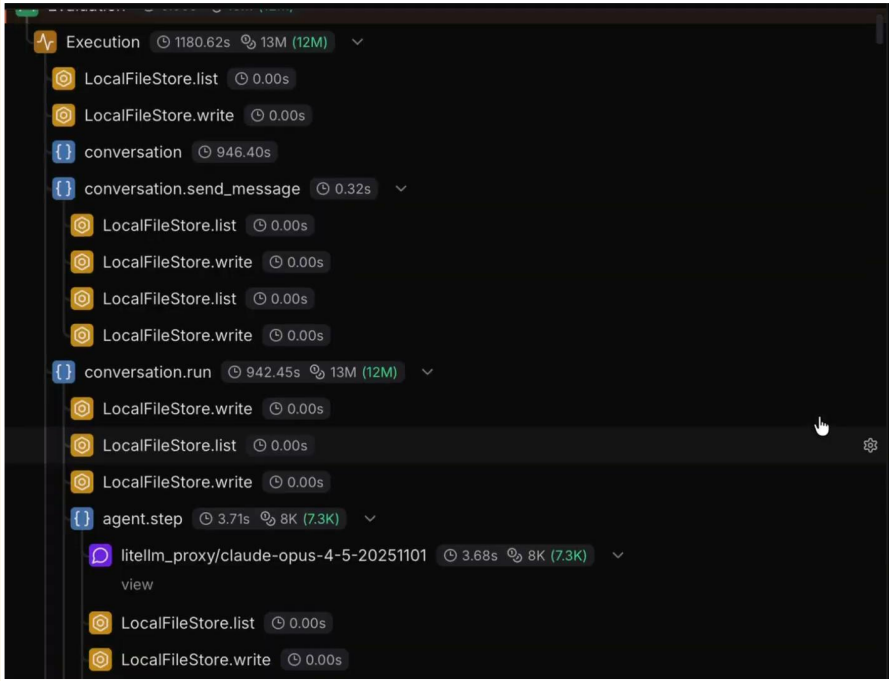
Tradeoffs:

- Capability
- Speed
- Cost

https://metr.org/

Why this takes 12M tokens

- Not one generation
- Hundreds of iterations
- Each step includes context + feedback
- The model isn't solving the problem. The system is.



https://laminar.sh/shared/evals/c97e4a45-8a14-428f-8eac-f77ef6eb75a8

Less can be more with agents

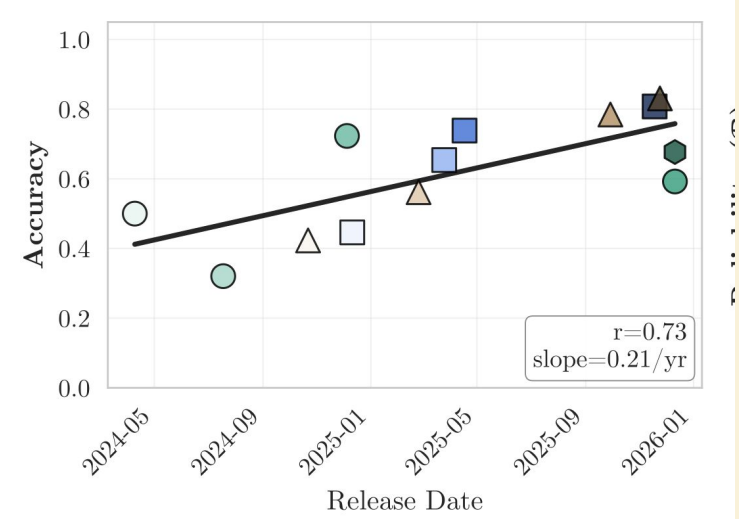
Minimal prompts force agents to explore and verify, rather than blindly following a hallucinated plan.

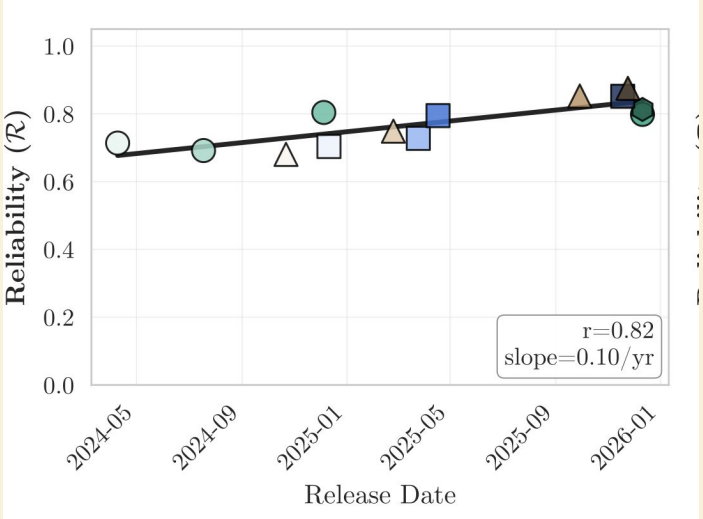
TABLE III: Test Pass Rates for Incremental and Clean State Development for Different Types of Prompts P1 and P2

Agent	Project	Test Pass Rate (%)					
		Incremental			Clean State		
		B	P1	P2	B	P1	P2
Claude Code	Piggymetrics	100.0	91.1	81.7	100.0	98.0	98.0
	Train-Ticket	100.0	92.6	60.6	81.5	97.0	97.0
	TeamSync	100.0	66.7	66.7	100.0	100.0	100.0
	Project-Management	92.3	44.3	43.9	92.3	92.6	96.3
	Avg. Agent Performance	-	**73.7**	**63.2**	-	**96.9**	**97.8**
Codex	Piggymetrics	100.0	93.8	93.3	100.0	99.0	65.7
	Train-Ticket	100.0	100.0	59.8	81.5	97.0	97.0
	TeamSync	100.0	66.7	0.0	100.0	100.0	100.0
	Project-Management	92.3	43.0	48.0	92.3	96.3	63.0
	Avg. Agent Performance	-	**75.9**	**50.3**	-	**98.1**	**81.4**
Code Qwen	Piggymetrics	100.0	66.7	84.1	100.0	98.0	98.0
	Train-Ticket	100.0	60.6	25.3	81.5	33.3	97.0
	TeamSync	100.0	33.3	33.3	100.0	100.0	100.0
	Project-Management	92.3	41.5	46.0	92.3	96.3	63.0
	Avg. Agent Performance	-	**50.5**	**47.2**	-	**81.9**	**89.5**

https://arxiv.org/pdf/2603.09004

Capability is rising faster than reliability.

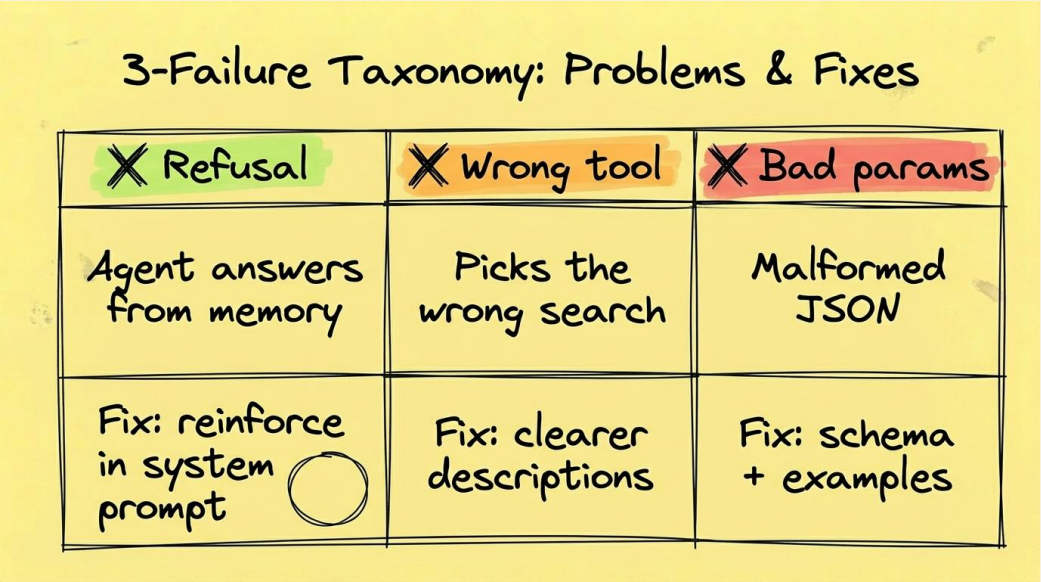




Models are getting better are writing code, but they are not doing it reliably, which is why we need a harness.

https://arxiv.org/pdf/2602.16666

TroubleShooting Tools



Many agent issues are tool-use problems.